MEMORANDUM

To:	Professor Brendon Anderson, Mechanical Engineering, California Polytechnic State University, bga@calpoly.edu
From:	Gabriel Choi, Carmen Alaichamy
Date:	May 27, 2026
Subject:	Lab 5: Sine Sweep Test and Closed-loop Position Control

---

Introduction

This memo summarizes results of Lab 5A and Lab 5B. The objective of this lab in part A was to implement a sine sweep test, to experimentally determine the open loop transfer function of the pendulum system. Then, for part B, the objective was to implement and tune a PID controller to optimize PID performance, and learn derivative controller drawbacks.

Methodology

LAB 5A

The experiment was conducted using an STM32 microcontroller connected to a DC motor and rotary encoder. A linear sine sweep input signal was applied to the pendulum system to excite a range of frequencies. The angular response of the pendulum was measured using the encoder and recorded in Jupyter Notebook. The recorded input and output data were used to generate experimental Bode plots for magnitude and phase. A second-order transfer function model was then tuned by adjusting ω_n, ζ, and K_{ss}until the theoretical response closely matched the experimental data.

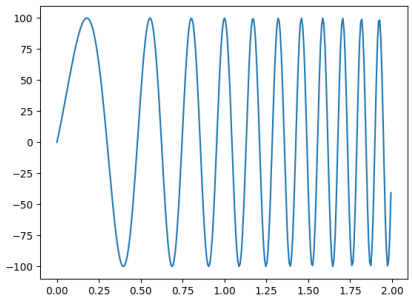


Figure 1. Linear Sine Sweep

LAB 5B

The experiment was conducted using an STM32 microcontroller connected to a DC motor and rotary encoder. A PID loop was implemented, tuning controller gains starting with Kp, then KD, and finally KI gains.

The angular response of the pendulum was measured using the encoder and recorded in Jupyter Notebook. The recorded input and output data were used to generate experimental Bode plots for magnitude and phase. The Jupyter Notebook file along with graphs are attached in Appendix A.

Results

LAB 5A

The following plots in Figure 2 shows the theoretical (red) and experimental (blue) Bode plots. The equation below shows the final transfer function, found from plotting the theoretical response over the experimental data.

$$\frac{0.018 * 6.7^2}{s^2 + 2 * 0.41 * 6.7s + 6.7^2}$$

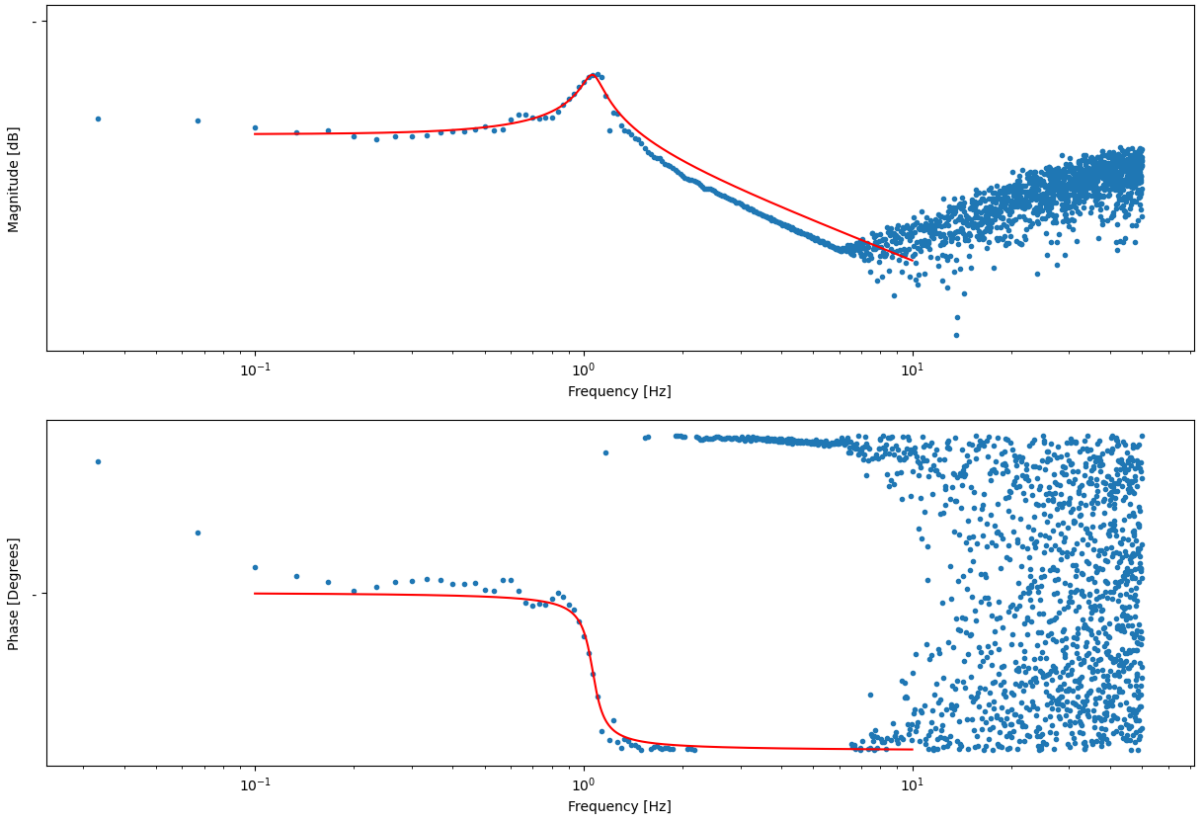


Figure 2. Experimental and Theoretical Overlayed Bode Plots

LAB 5B

The following step response and controller output plot in Figure 3 shows the optimal configuration after tuning each of the gains in the PID controller. This plot shows that steady state error was eliminated by the integral controller, while transient response was improved by decreasing settling time and decreasing number of oscillations using the derivative controller. Finally, we ensured that the controller output was under the saturation limit.

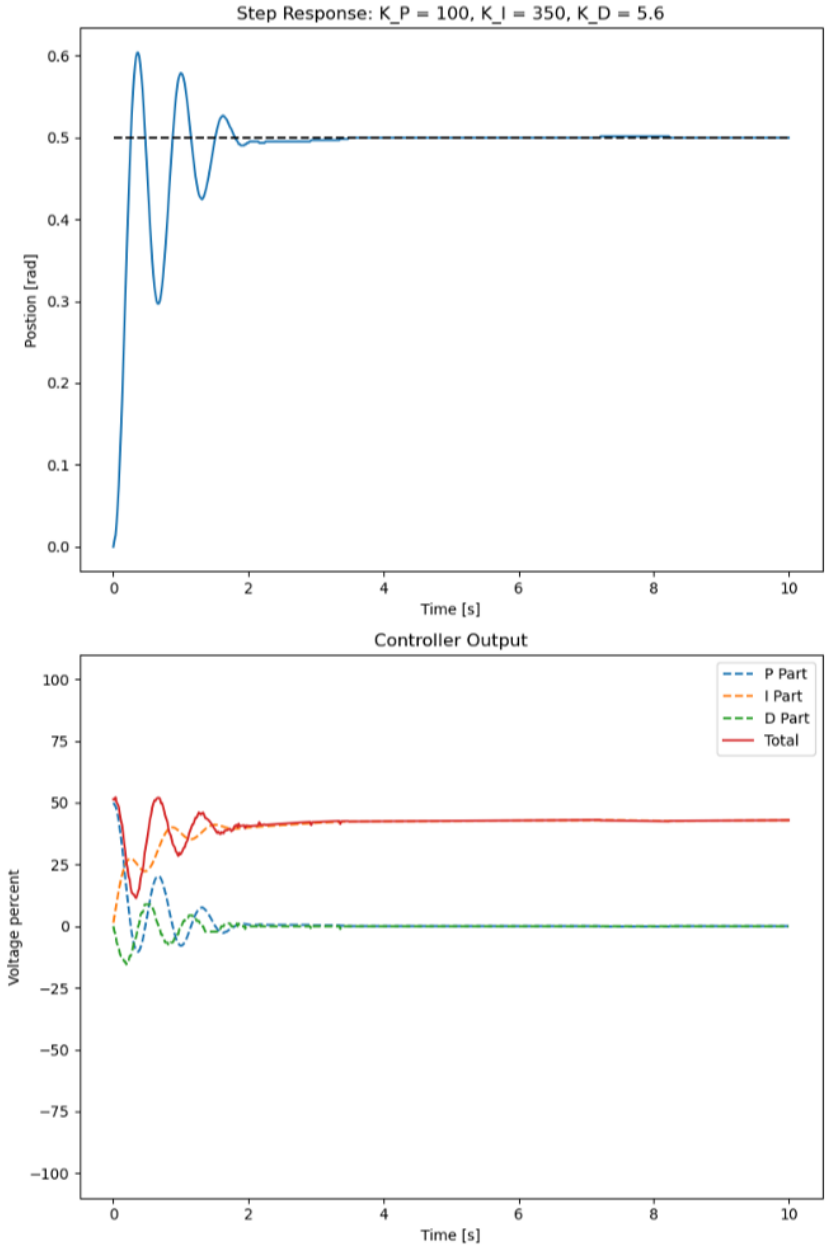


Figure 3. Step response and controller output using optimal PID gains, where Kp = 100, KI = 350, and KD = 5.6.

Next, to validate our experimental system, we calculated z1 and z2 in Matlab. This calculation is shown below in Figure 4.

```
K_p = 100;
K_i = 350;
K_d = 5.6;

z_1 = ((-K_p/K_d) + sqrt((K_p/K_d)^2 - (4*(K_i/K_d)))) / 2;
z_2 = ((-K_p/K_d) - sqrt((K_p/K_d)^2 - (4*(K_i/K_d)))) / 2;

disp(z_1)
```

```
   -4.7789
```

```
disp(z_2)
```

```
   -13.0782
```

Figure 4. Calculating z1 and z2.

Finally, we graphed the root locus and response in Matlab using rltool with added z1 and z2 zeros, and pole at origin. The root locus is shown in Figure 5, and the response is shown in Figure 6.

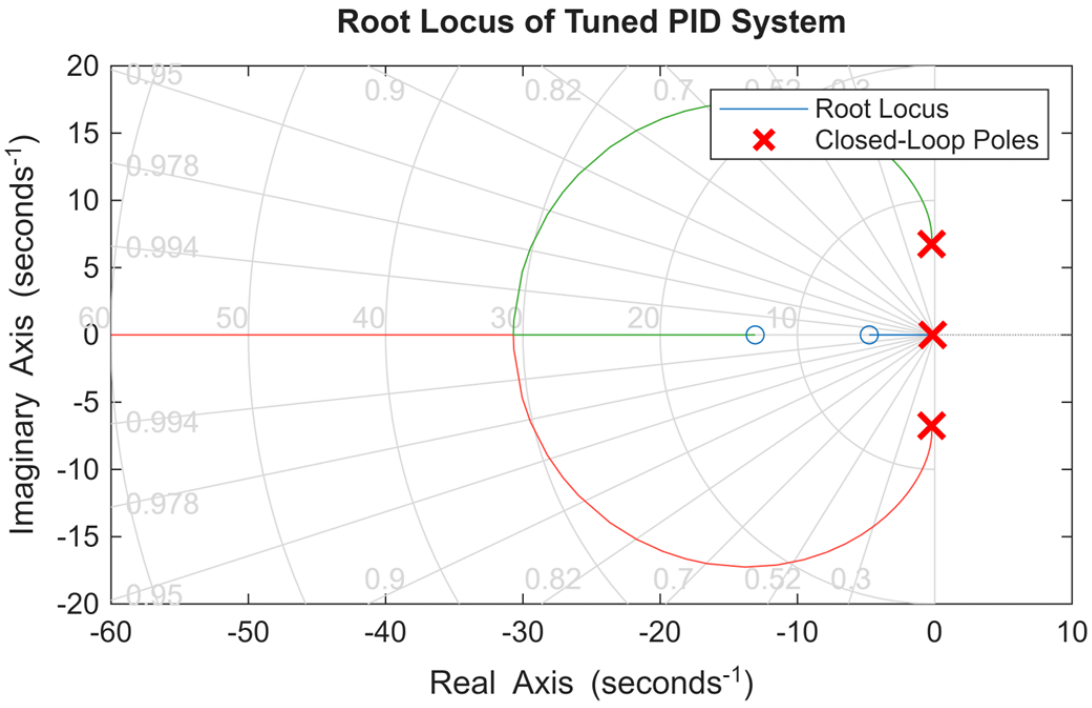


Figure 5. Root Locus With Optimized Gain Values Kp=100, Ki=350, Kd=5.6

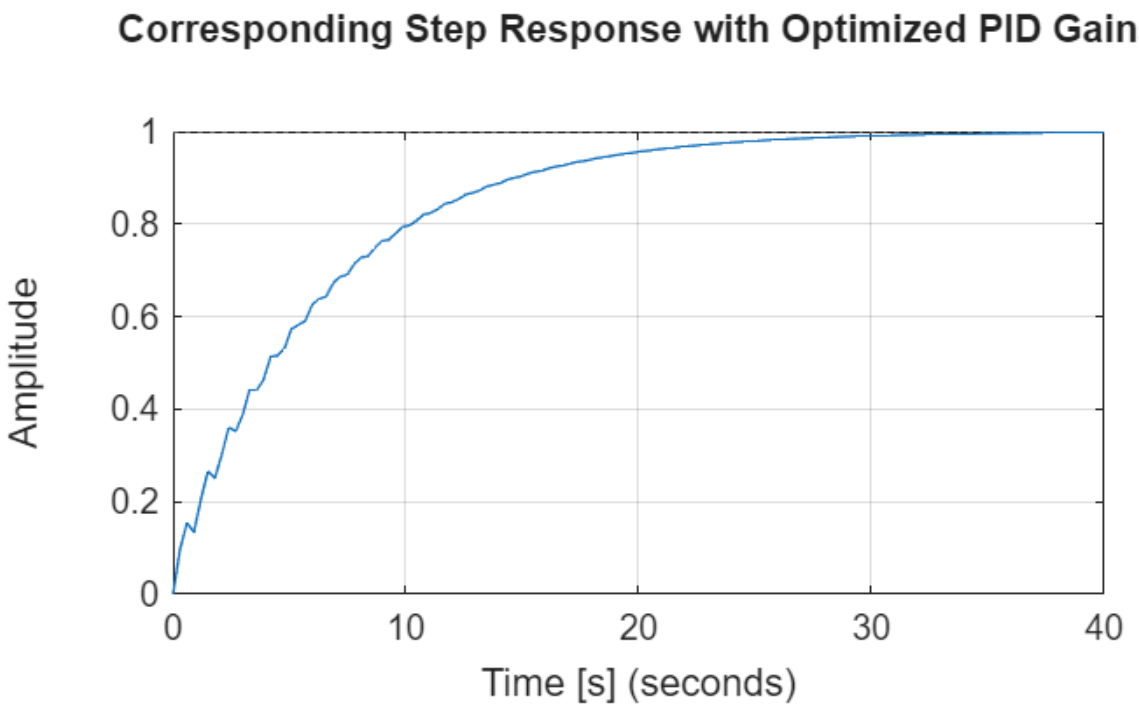


Figure 6. Time Step Response for Gain Values of Kp=100, Ki=350, Kd=5.6

Discussion

LAB 5A

Justify how ωn, ζ, and Kss affect the shape of the Bode plots.

K_{ss} affects the overall gain of the system and vertically shifts the magnitude plot. Increasing K_{ss}raises the magnitude response across all frequencies while decreasing K_{ss} lowers the response.

The damping ratio ζcontrols the sharpness of the resonance peak. Lower damping produces a larger resonant peak and greater oscillatory behavior, while higher damping smooths the response and reduces overshoot.

The natural frequency $\omega_n$determines the location of the resonance peak and shifts the frequency response horizontally along the frequency axis.

What frequency range can we consider reliable in the experiment? Hint: Check the frequency range of the sweep signal.

The frequency range that we consider reliable for this experiment is within the range of 2-6 hz for the long pendulum. This is because outside of that range we would encounter constructive interference which would cause our pendulum to oscillate outside of the range of $-\pi < \theta < \pi$.

Why does the data look very chaotic after 10Hz? How can we improve the frequency response after 10HZ?

After 10 Hz, the data becomes chaotic because the pendulum and motor cannot respond accurately to very fast input signals. At high frequencies, sensor noise, sampling limitations, and mechanical vibrations become more significant, making the measurements unreliable. The frequency response could be improved by increasing the sampling rate, using better filtering techniques, and improving the hardware or sensor accuracy.

LAB 5B

Effects of P, I, and D Control on System Response

The proportional controller increased the speed of the response and reduced the overall error by applying a correction proportional to the position error. As the proportional gain increased, the system responded faster, but excessive proportional gain caused larger oscillations and overshoot. The optimal P control response is shown in Figure 7a.

The derivative controller improved the damping of the system by responding to how quickly the error changed over time. This reduced overshoot and helped stabilize the pendulum more quickly. However, high derivative gains amplified measurement noise from the encoder, which introduced vibrations into the controller output. As shown in Figure 7b, the response has less oscillation, faster settling time, but still large steady state error.

Finally, the integral controller minimized the steady-state error by accumulating the error over time and continuously correcting small remaining offsets from the setpoint. While this improved long-term accuracy, excessive integral gain increased oscillation and overshoot because the stored accumulated error continued driving the system even after the pendulum approached the desired position. The PID controller response above in Figure 3 shows integral controller benefits, where it is clear that in addition to the benefits of the derivative controller, steady state error is eliminated.

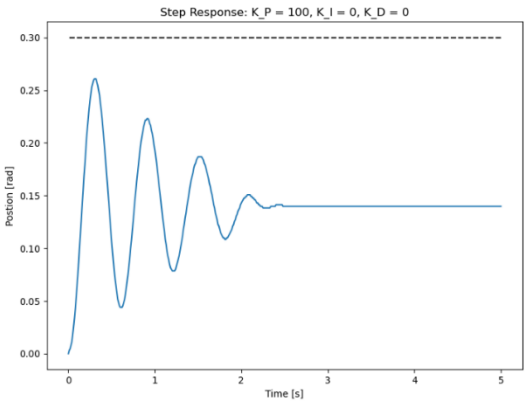


Figure 7a. (Left) P control response.

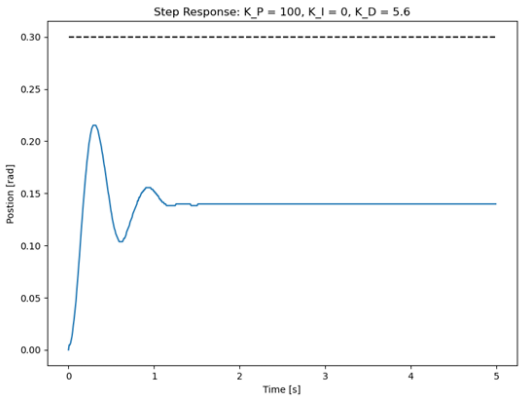


Figure 7b. (Right) PD control response.

Effect of Changing the Sampling Rate

The sampling period was increased from 5000 μs to 10000 μs during the experiment. Increasing the sampling period reduced the effective sampling frequency of the controller. This helped reduce vibrations because the derivative controller became less sensitive to high-frequency encoder noise. As a result, resonance effects near the pendulum's natural frequency were reduced, producing a smoother controller output and a more stable response.

Differences Between Experimental and Simulated Responses

The simulated and experimental responses differed because the mathematical model used in simulation could not perfectly represent the real hardware system. The simulation assumed ideal system behavior, while the physical pendulum experienced friction, electrical noise, motor saturation, sensor inaccuracies, and small timing delays.

Additionally, the encoder measurements introduced noise into the feedback signal, which became more noticeable when derivative control was applied. The transfer function parameters used in the simulation were also estimated values rather than exact physical measurements, so the simulated response could only approximate the real experimental behavior.

Conclusion

The part A Lab successfully characterized the frequency response of the pendulum system using experimental Bode plots. A second-order transfer function model was developed that reasonably matched the measured magnitude and phase responses. The lab demonstrated how ω_n, ζ, and K_{ss}affect system dynamics and frequency-domain behavior.

In part B, the effects of each gain in a PID controller was investigated, and we successfully tuned a PID controller using the experimental pendulum set-up.

Appendix A

This cell will import all of the necessary libraries for this lab. These include the pyserial library which is used to communicate with the microcontroller, NumPy which is used to load data from a CSV file, and pyplot from matplotlib which is a plotting package with almost identical commands to MATLAB. Run this cell first.

In [5]:
```
from serial import Serial
import time
import numpy as np
from matplotlib import pyplot as plt
```

This cell defines a function which will plot the step response. No need to modify this cell.

In [6]:
```
def plot_step(kp, ki, kd, step_pos, filename):
    # Import the data from the file
    data = np.genfromtxt(filename, delimiter=',')
    times = data[:,0]
    positions = data[:,1]
    percents = data[:,2]
    p_percents = data[:,3]
    i_percents = data[:,4]
    d_percents = data[:,5]

    # print(get_stats(times, positions, step_pos))

    # Make a figure and set the size to 6x6 in
    plt.rcParams['figure.figsize'] = (8, 12)
    plt.figure()

    # Plot the step response of position vs time on the top half
    plt.subplot(211)
    # plt.figure()
    plt.plot(times, positions)
    plt.hlines(step_pos, times[1], times[-1], colors='k', linestyles='dashed')
    plt.xlabel("Time [s]")
    plt.ylabel("Postion [rad]")
    plt.title(f'Step Response: K_P = {kp}, K_I = {ki}, K_D = {kd}')

    # Plot the proportional and integral components on the bottom half
    plt.subplot(212)
    plt.plot(times, p_percents, '--')
    plt.plot(times, i_percents, '--')
    plt.plot(times, d_percents, '--')
    plt.plot(times, percents)
    plt.ylim(-110, 110)
    plt.xlabel("Time [s]")
    plt.ylabel("Voltage percent")
    plt.title("Controller Output")
    plt.legend(['P Part', 'I Part', 'D Part', 'Total'])
    plt.tight_layout()
    plt.show()
```

This cell defines a function which will connect to the microcontroller over the serial port. It will send the desired gains, velocity setpoint to step to, and total time to run the test for, as specified in the variables. Then it will collect the data and write it to `filename`. Finally it will plot the step response using the previous function. No need to modify this cell.

In [7]:
```
def run_step(kp, ki, kd, step_pos, t_step, filename, out_max=100):
    # Uncomment this line and give the name of the serial port which the Nucleo
    # is connected to, as a string. This should be something like
    # 'COMx' for Windows and
    # '/dev/tty.usbmodemXXXX' for Mac/Linux
    # Windows: Open Device Manager and look under "Ports (COM & LPT)" for a
    # "USB Serial Device"
    # Mac/Linux: Open terminal and run "ls /dev/tty.*" to find the port matching th
    # format above.
    serport = 'COM6'

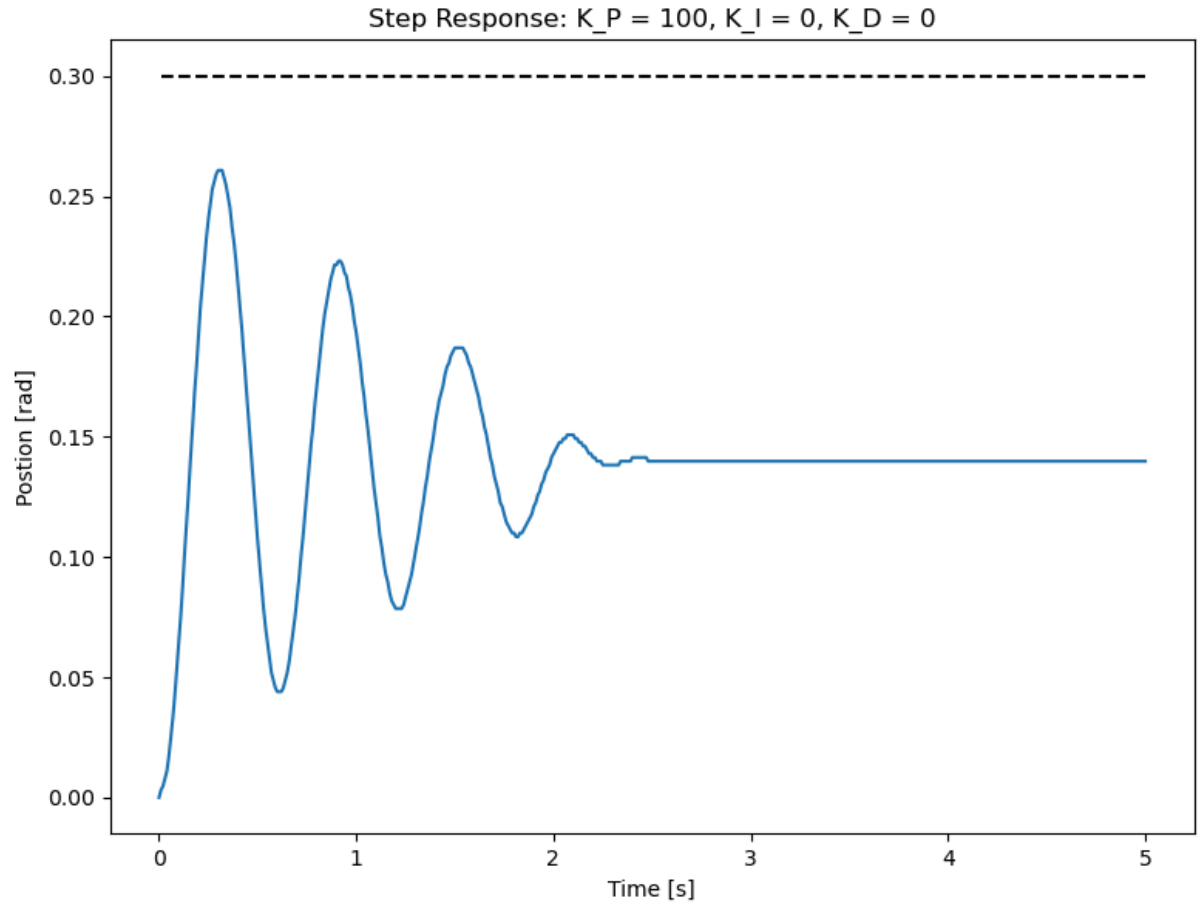
    # Open the serial port, and give it the name ser.
    with Serial(serport, baudrate=115200, timeout=5) as ser:
        # Restart the microcontroller
        ser.write(b'\x02\x04')
        # Write the parameters to the microcontroller
        ser.write((str(kp) + '\r\n').encode())
        ser.write((str(ki) + '\r\n').encode())
        ser.write((str(kd) + '\r\n').encode())
        ser.write((str(step_pos) + '\r\n').encode())
        ser.write((str(out_max) + '\r\n').encode())
        ser.write((str(t_step) + '\r\n').encode())

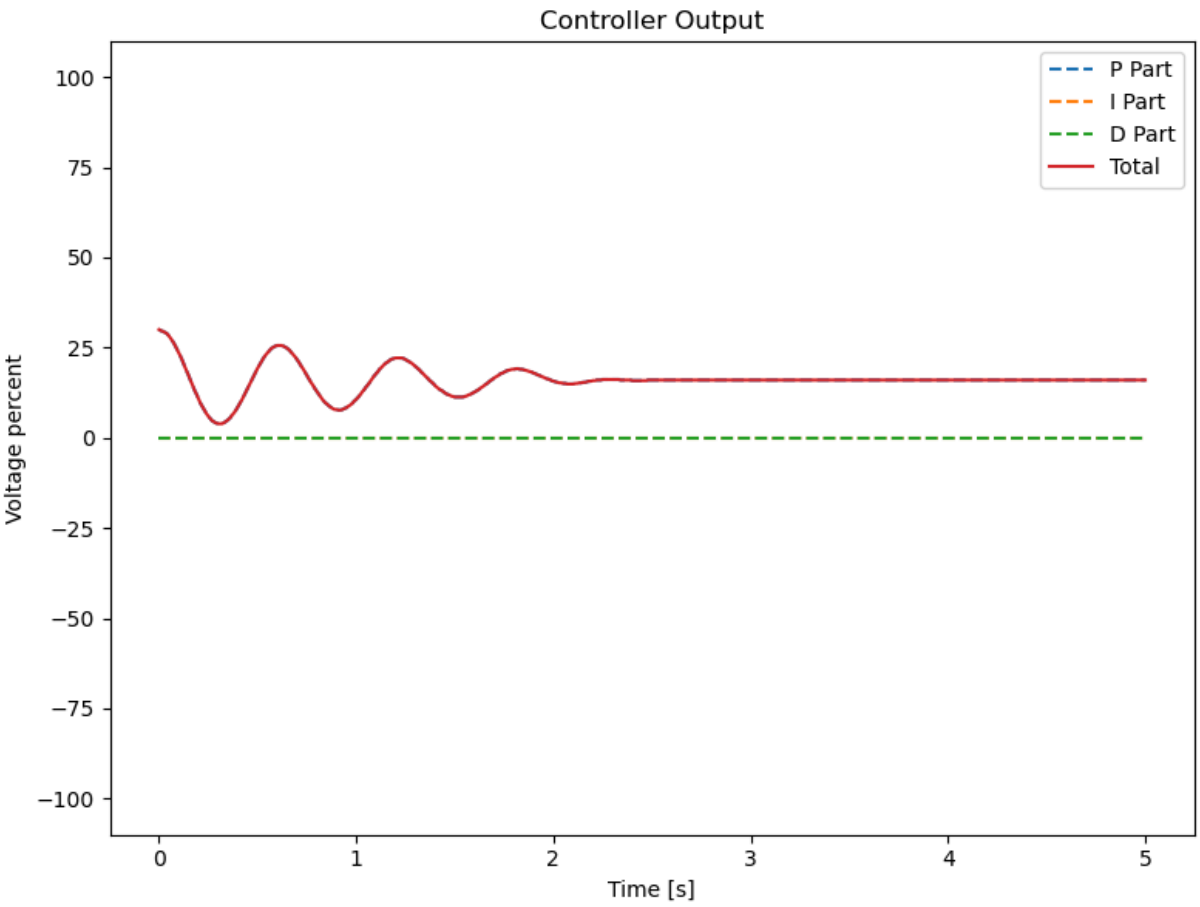
        # Open the data file for writing, and give it the name datafile
        with open(filename, 'w') as f:
            while True:
                # Read the current line and decode from bytes to a string
                line = ser.readline().decode()
                # Print the line so we can see what the microcontroller is doing
                # print(line, end='')
                # If the line starts with END, then we know the data is complete
                if line[:3] == 'END':
                    break
                # If the line has a comma, it is a line of CSV data.
                elif ',' in line:
                    f.write(line.strip() + '\n')

    plot_step(kp, ki, kd, step_pos, filename)
```

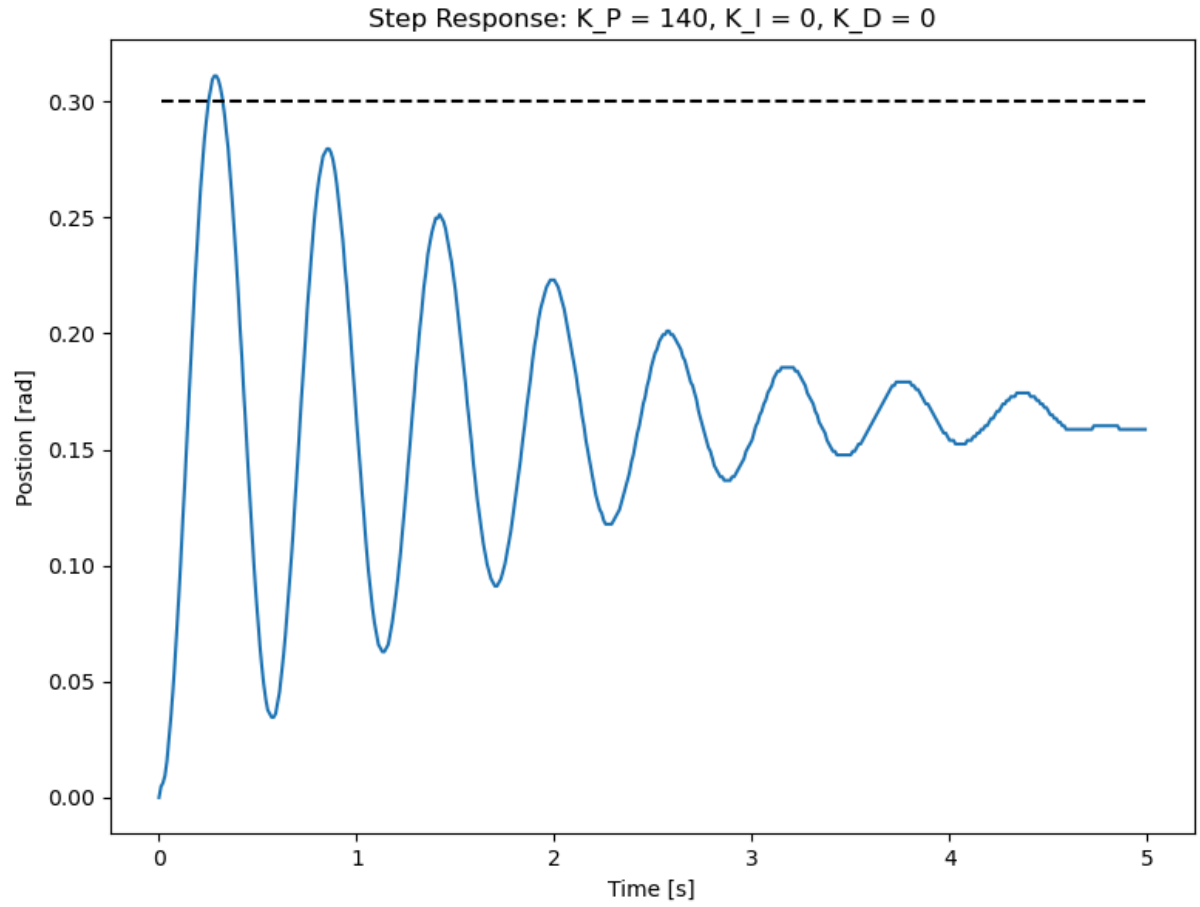
Make several cells here to call `run_step` for each of the required gains from the lab manual. Use a unique filename for the data files.

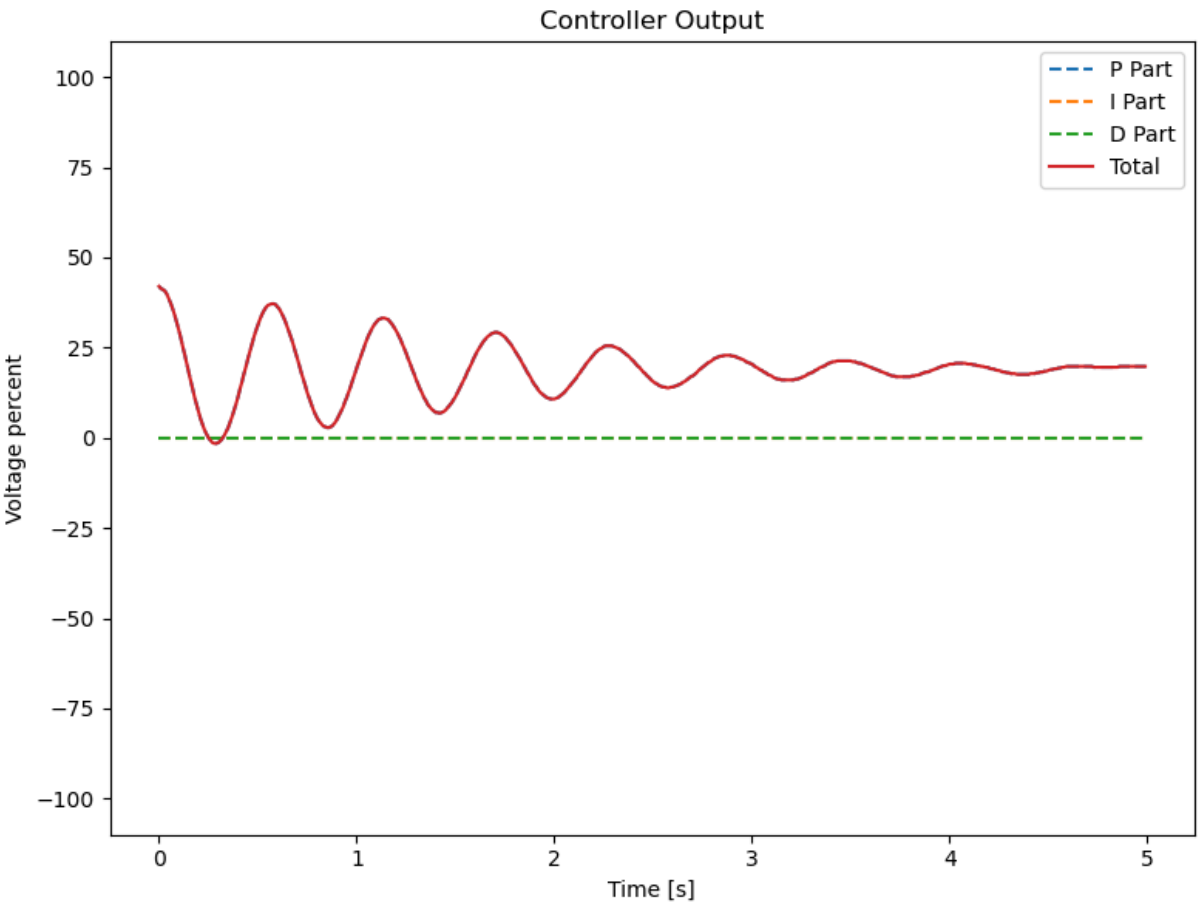
In [17]:
```
"run_step(100, 0, 0, 0.3, 5000, "data1.csv")"
```


Step Response: K_P = 100, K_I = 0, K_D = 0
Postion [rad]
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
1
2
3
4
5
Time [s]

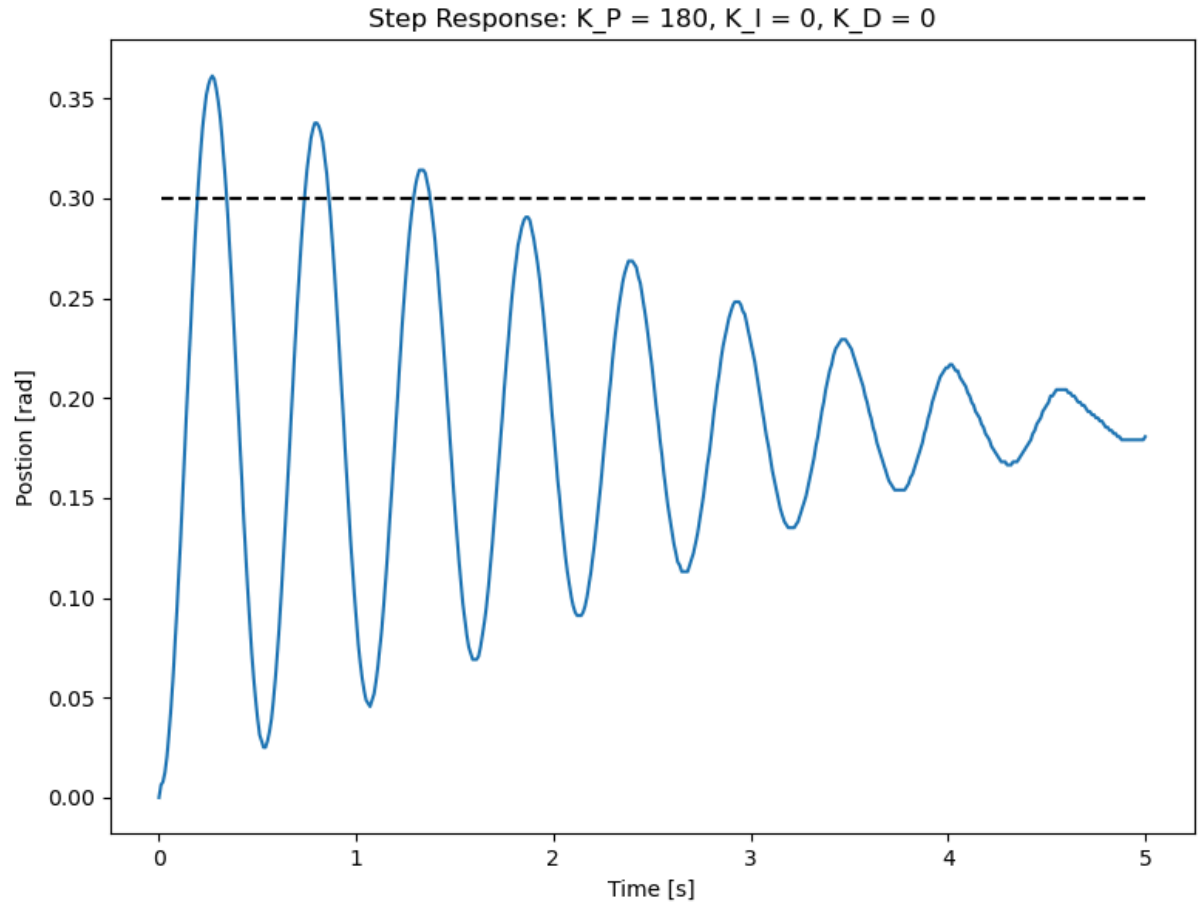

Controller Output
P Part
I Part
D Part
Total
Voltage percent
100
75
50
25
0
−25
−50
−75
−100
0
1
2
3
4
5
Time [s]

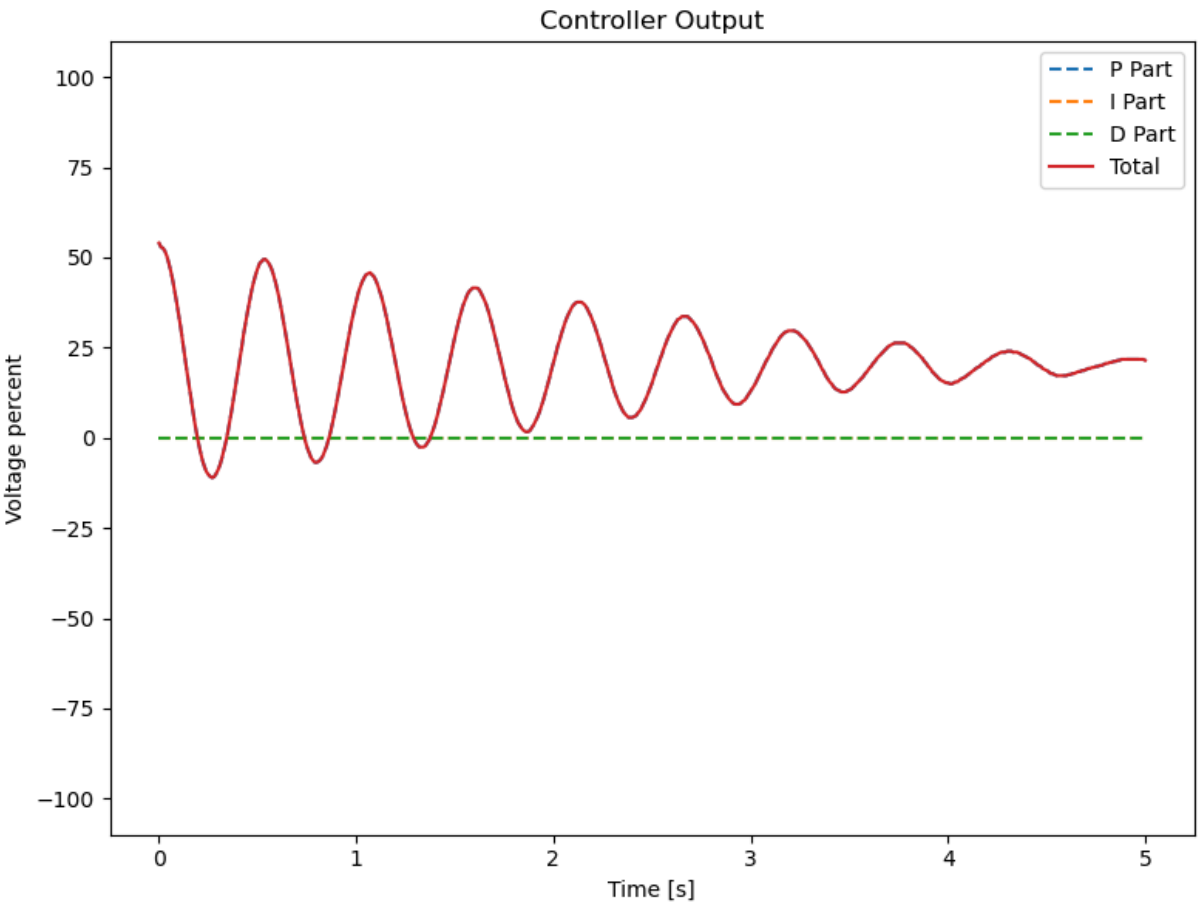
In [18]:
```
"run_step(140, 0, 0, 0.3, 5000, "data1.csv")"
```


Step Response: K_P = 140, K_I = 0, K_D = 0
Postion [rad]
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
1
2
3
4
5
Time [s]

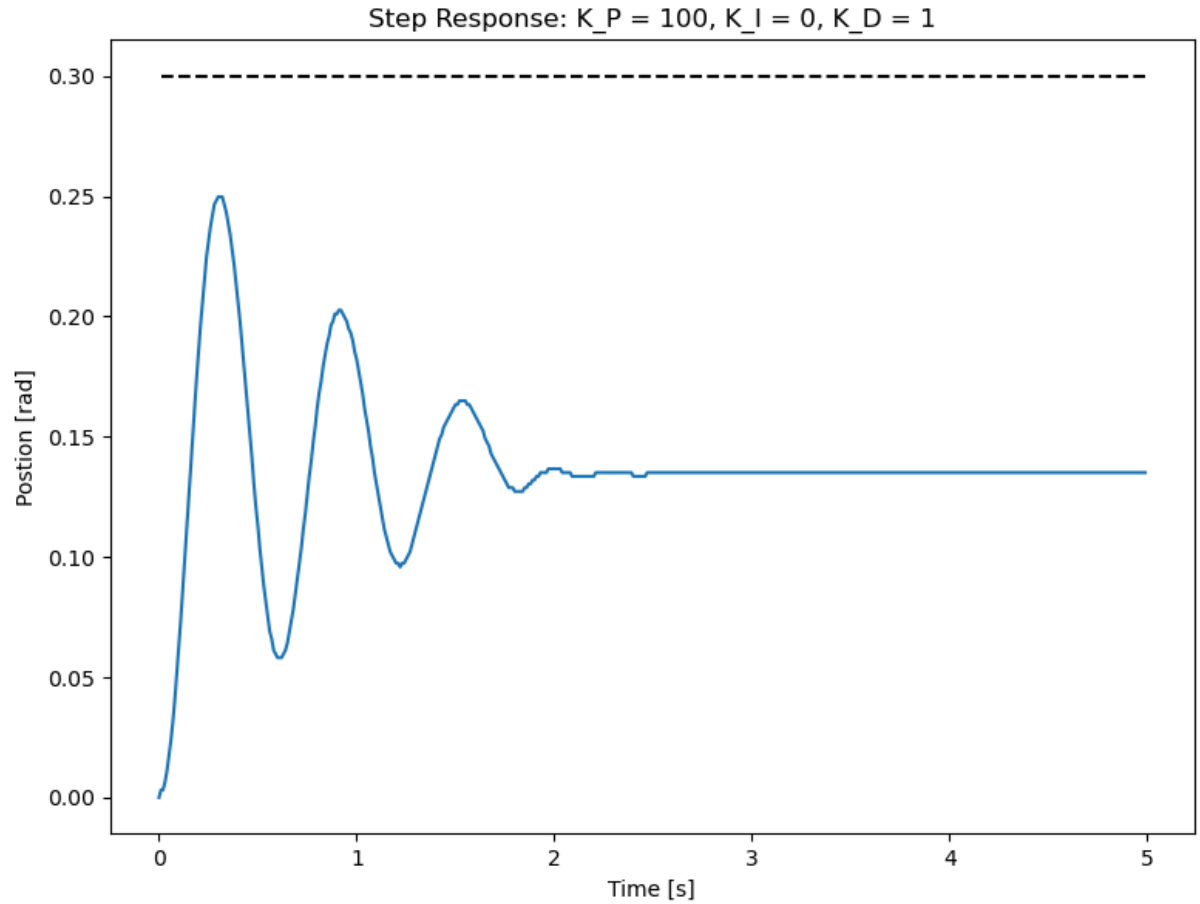

Controller Output
P Part
I Part
D Part
Total
Voltage percent
100
75
50
25
0
−25
−50
−75
−100
0
1
2
3
4
5
Time [s]

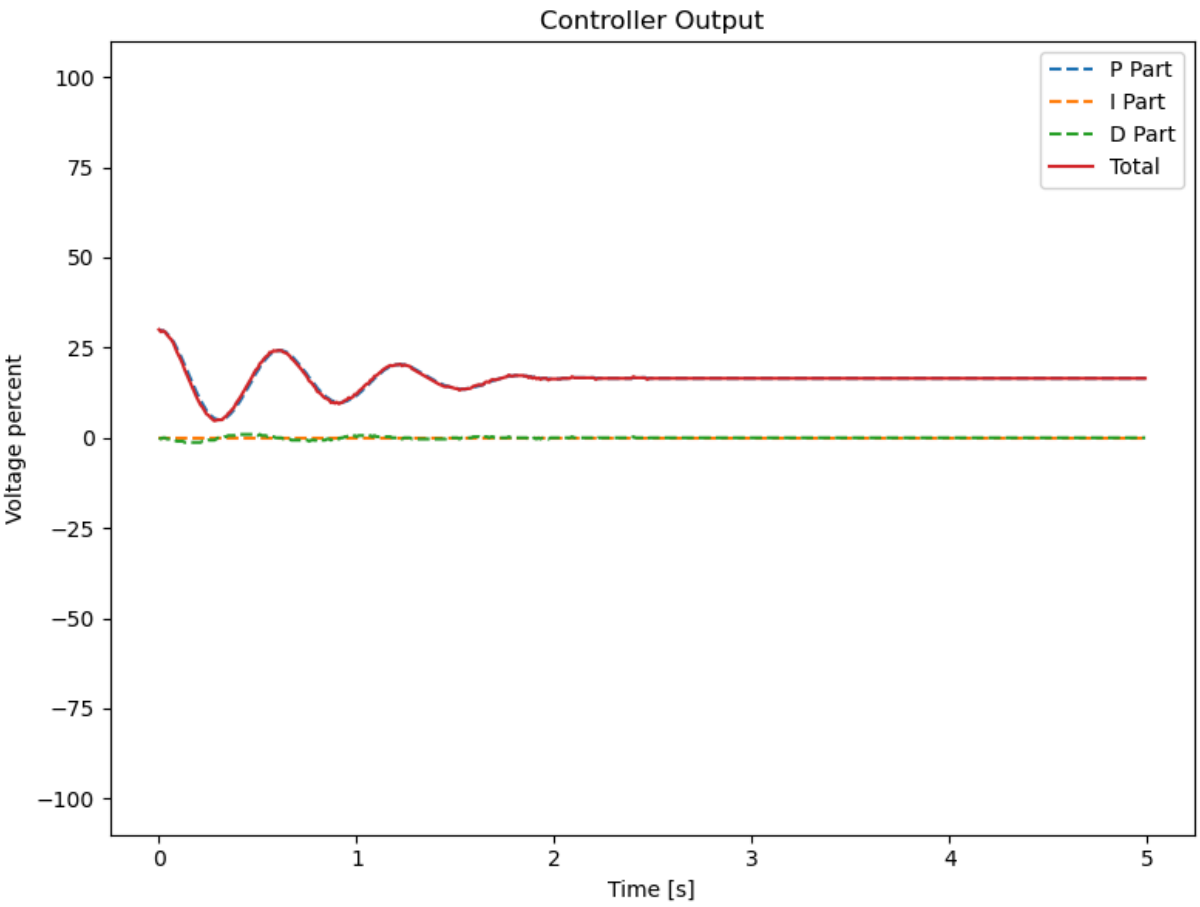
In [19]:
```
"run_step(180, 0, 0, 0.3, 5000, "data1.csv"
```


Step Response: K_P = 180, K_I = 0, K_D = 0
Postion [rad]
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
1
2
3
4
5
Time [s]

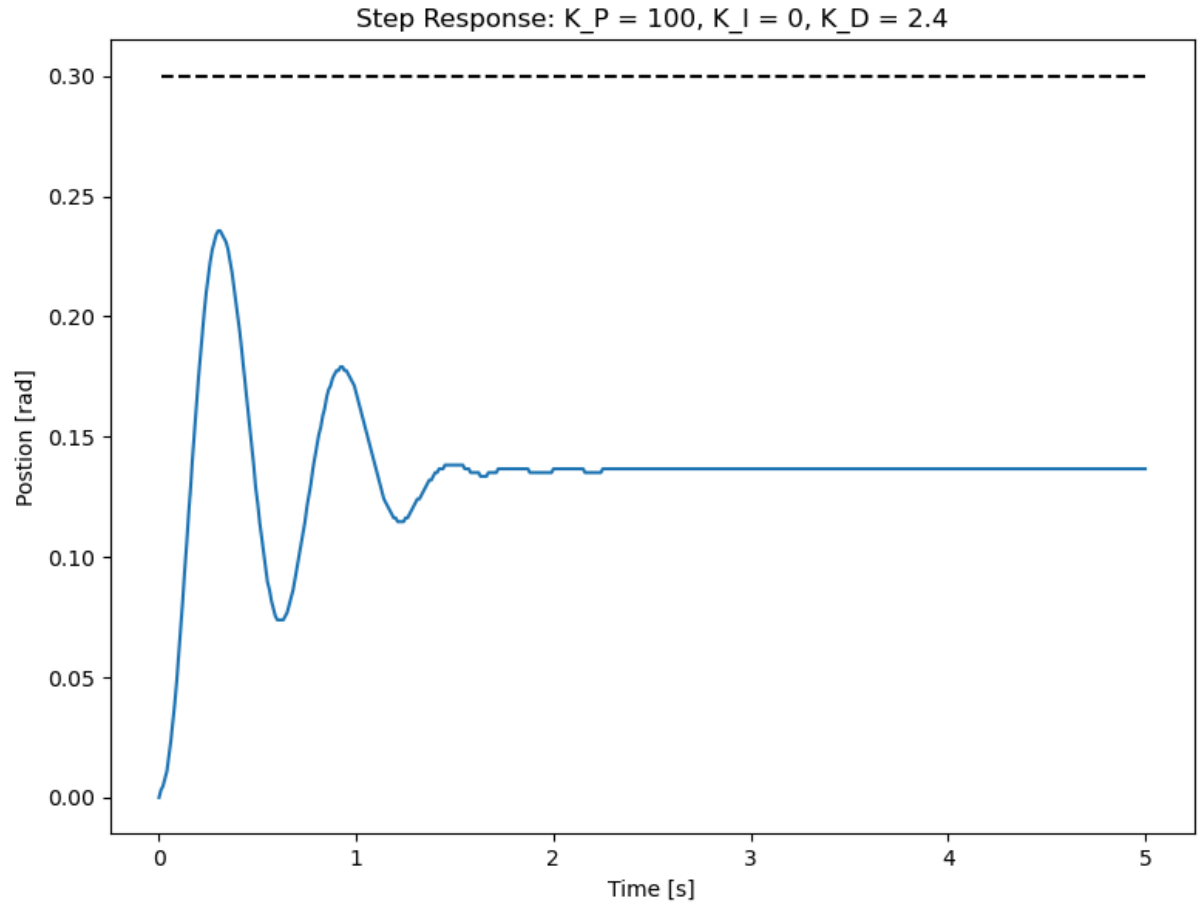

Controller Output
P Part
I Part
D Part
Total
Voltage percent
100
75
50
25
0
−25
−50
−75
−100
0
1
2
3
4
5
Time [s]

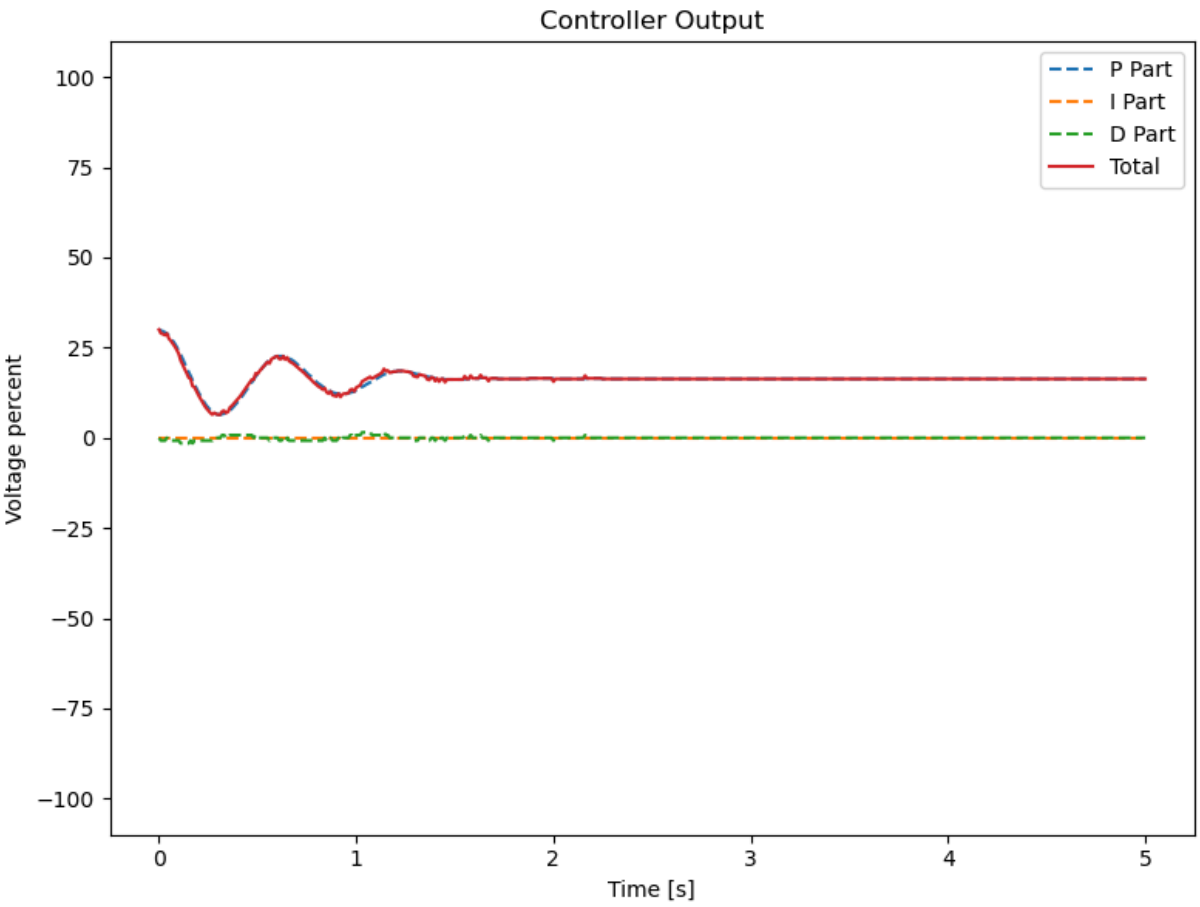
In [20]:
```
"run_step(100, 0, 1, 0.3, 5000, "data1.csv")"
```


Step Response: K_P = 100, K_I = 0, K_D = 1
Postion [rad]
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
1
2
3
4
5
Time [s]

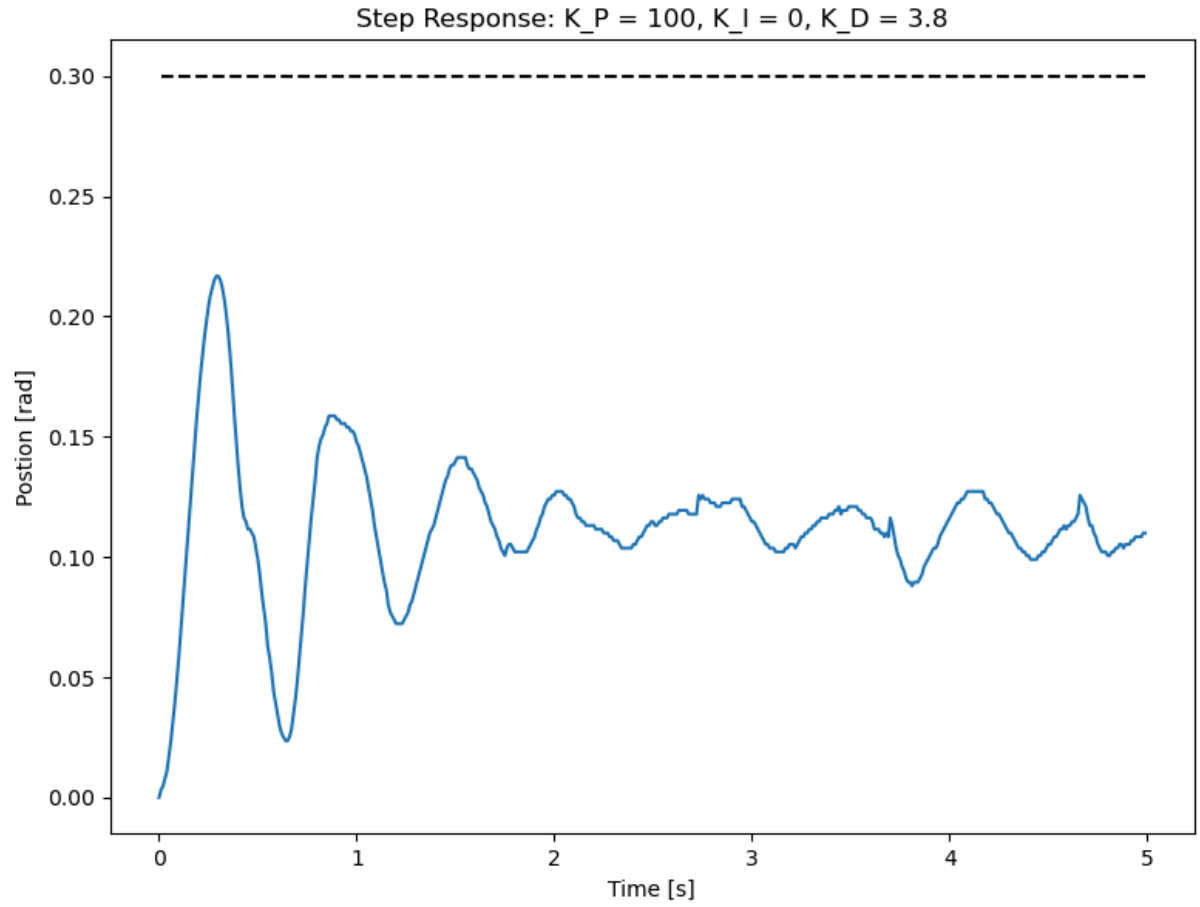

Controller Output
P Part
I Part
D Part
Total
Voltage percent
100
75
50
25
0
−25
−50
−75
−100
0
1
2
3
4
5
Time [s]

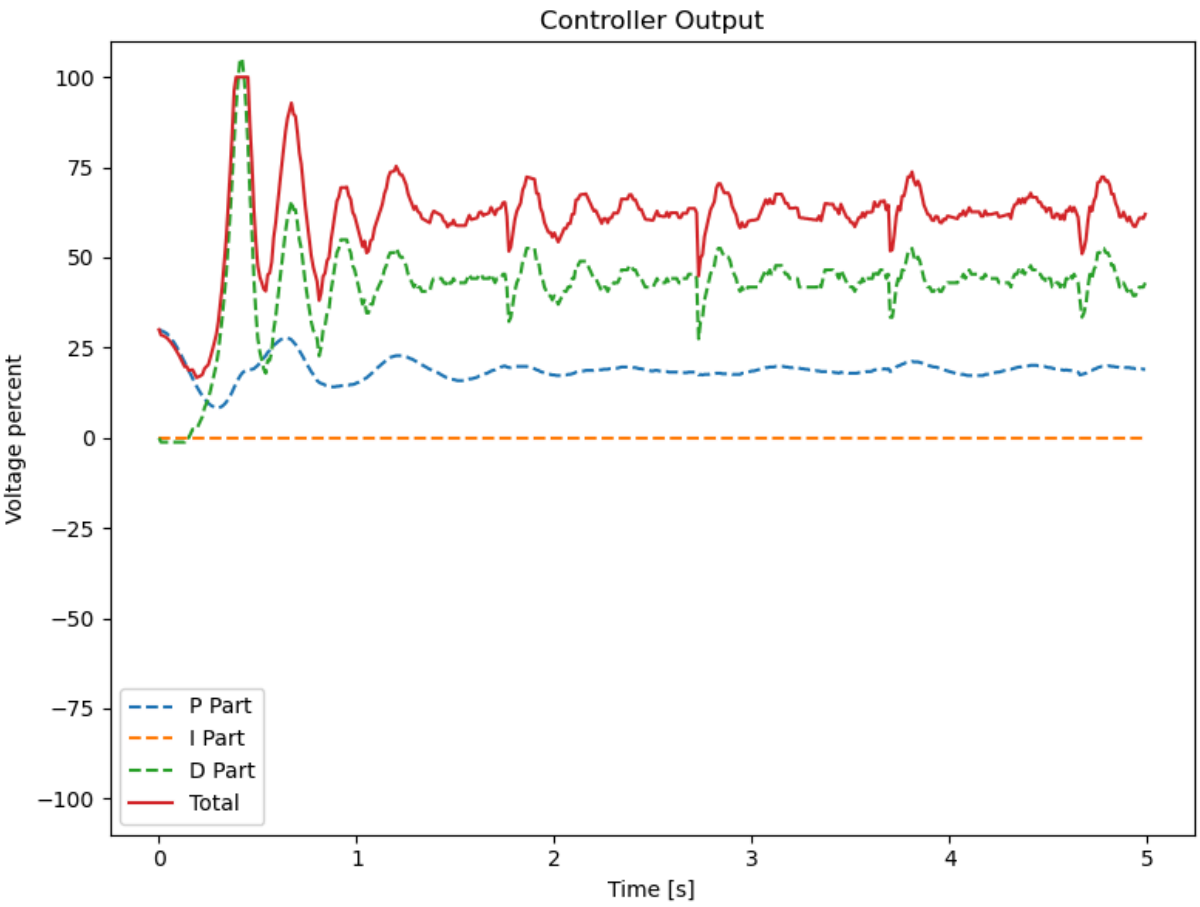
In [21]:
```
"run_step(100, 0, 2.4, 0.3, 5000, "data1.csv")"
```


Step Response: K_P = 100, K_I = 0, K_D = 2.4
Postion [rad]
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
1
2
3
4
5
Time [s]

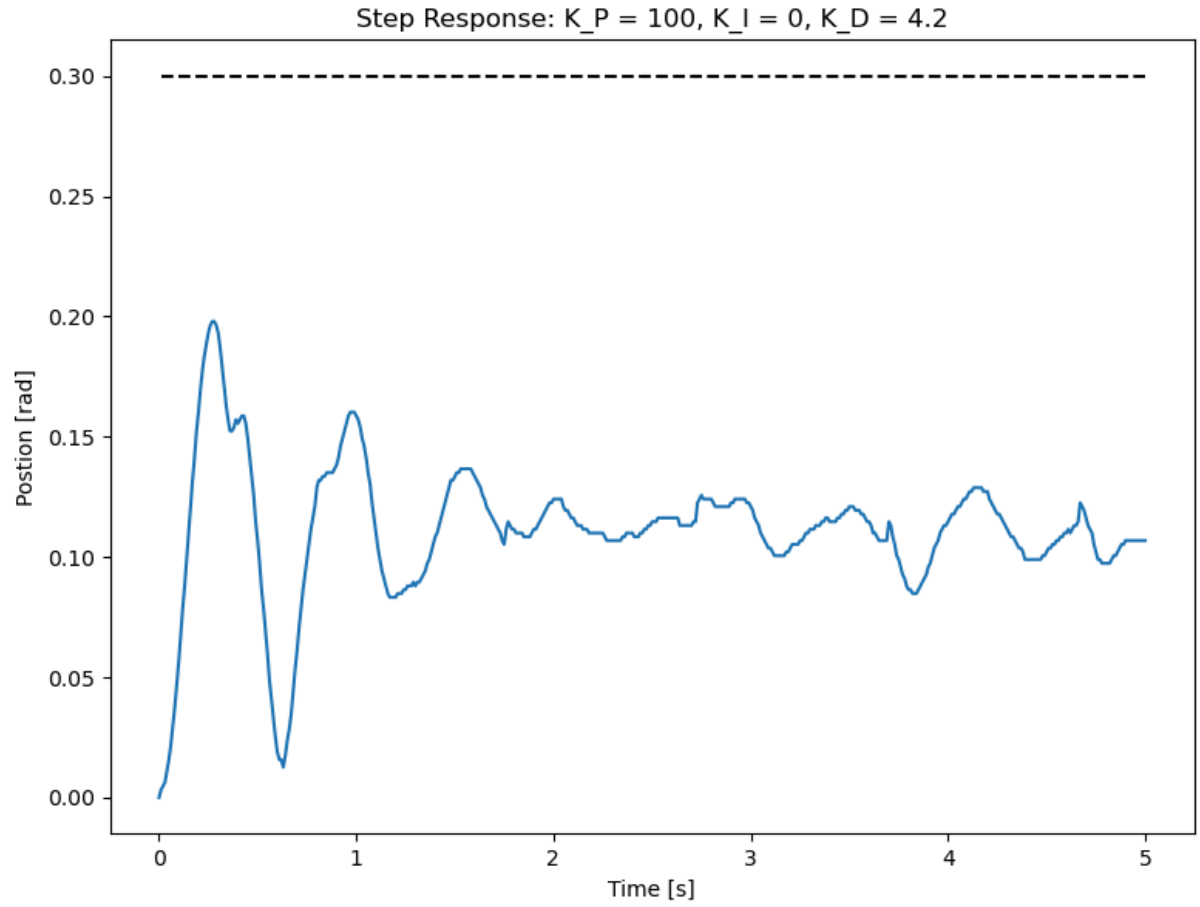

Controller Output
P Part
I Part
D Part
Total
Voltage percent
100
75
50
25
0
−25
−50
−75
−100
0
1
2
3
4
5
Time [s]

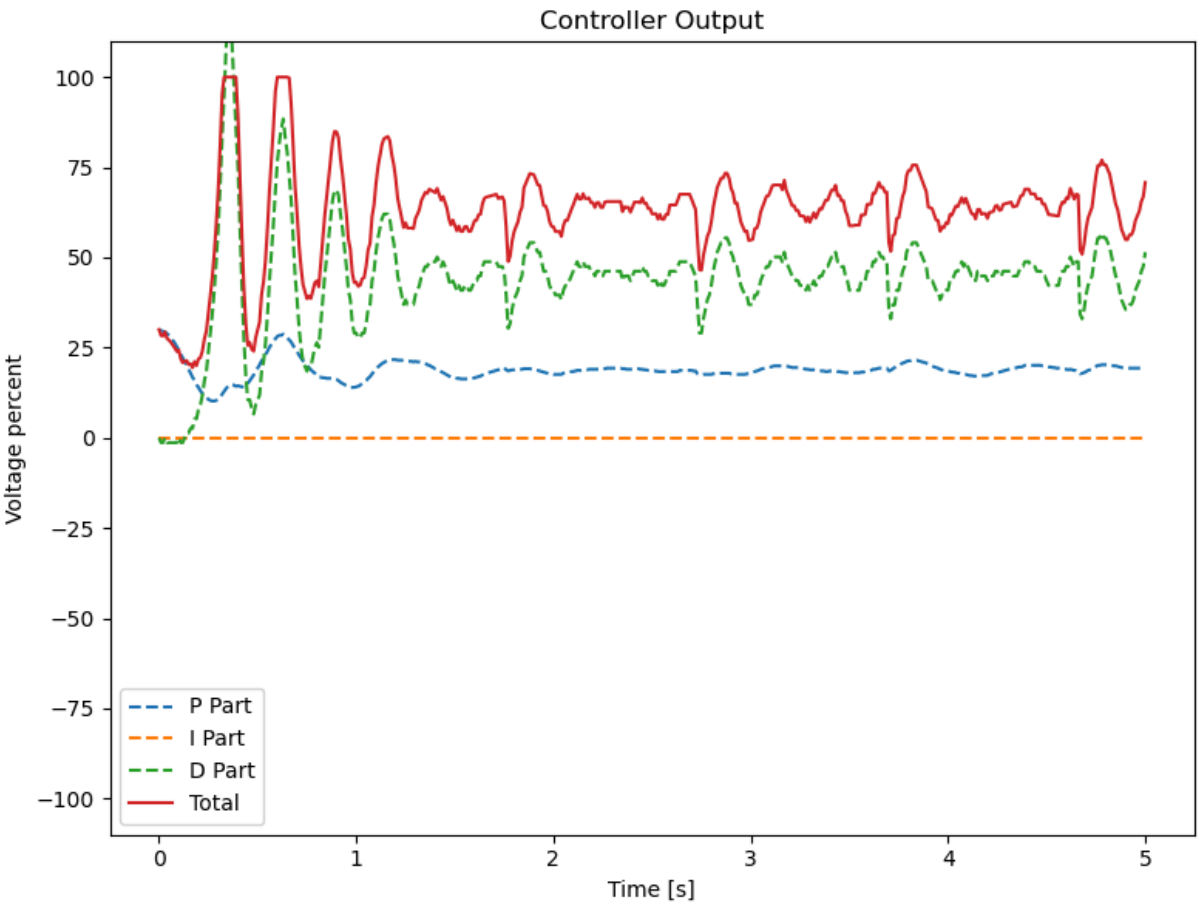
In [22]:
```
"run_step(100, 0, 3.8, 0.3, 5000, "data1.csv")"
```


Step Response: K_P = 100, K_I = 0, K_D = 3.8
Postion [rad]
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
1
2
3
4
5
Time [s]

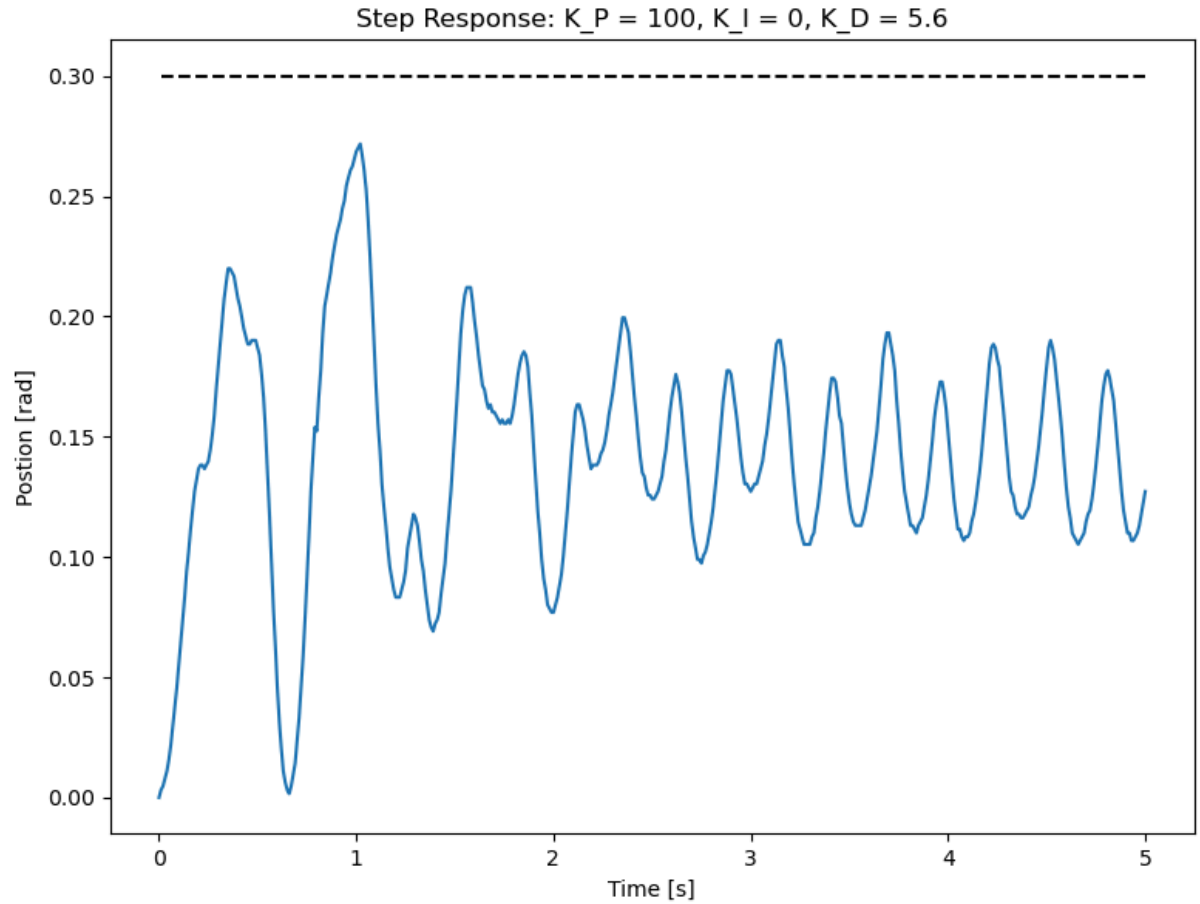

Controller Output
Voltage percent
100
75
50
25
0
−25
−50
−75
−100
P Part
I Part
D Part
Total
0
1
2
3
4
5
Time [s]

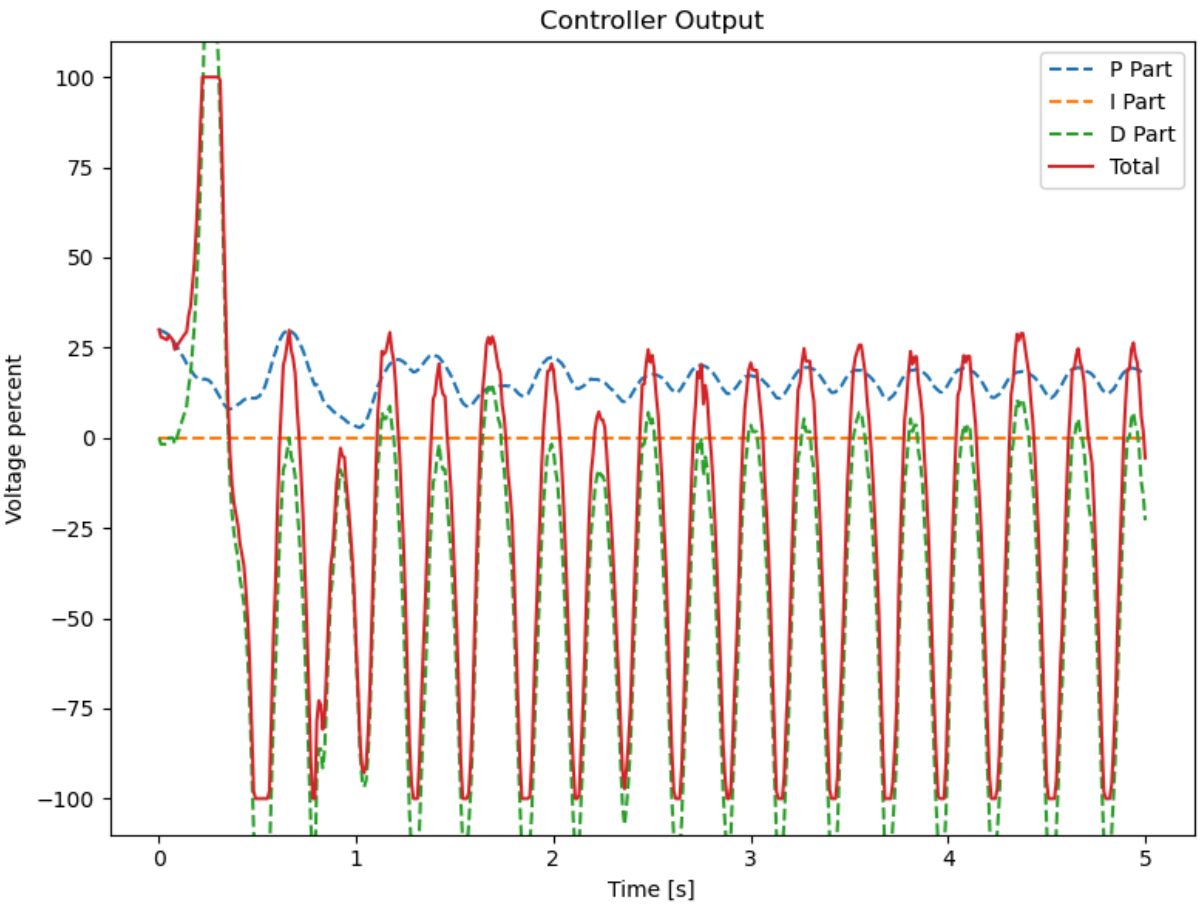
In [23]:
```
"run_step(100, 0, 4.2, 0.3, 5000, "data1.csv")"
```


Step Response: K_P = 100, K_I = 0, K_D = 4.2
Postion [rad]
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
1
2
3
4
5
Time [s]

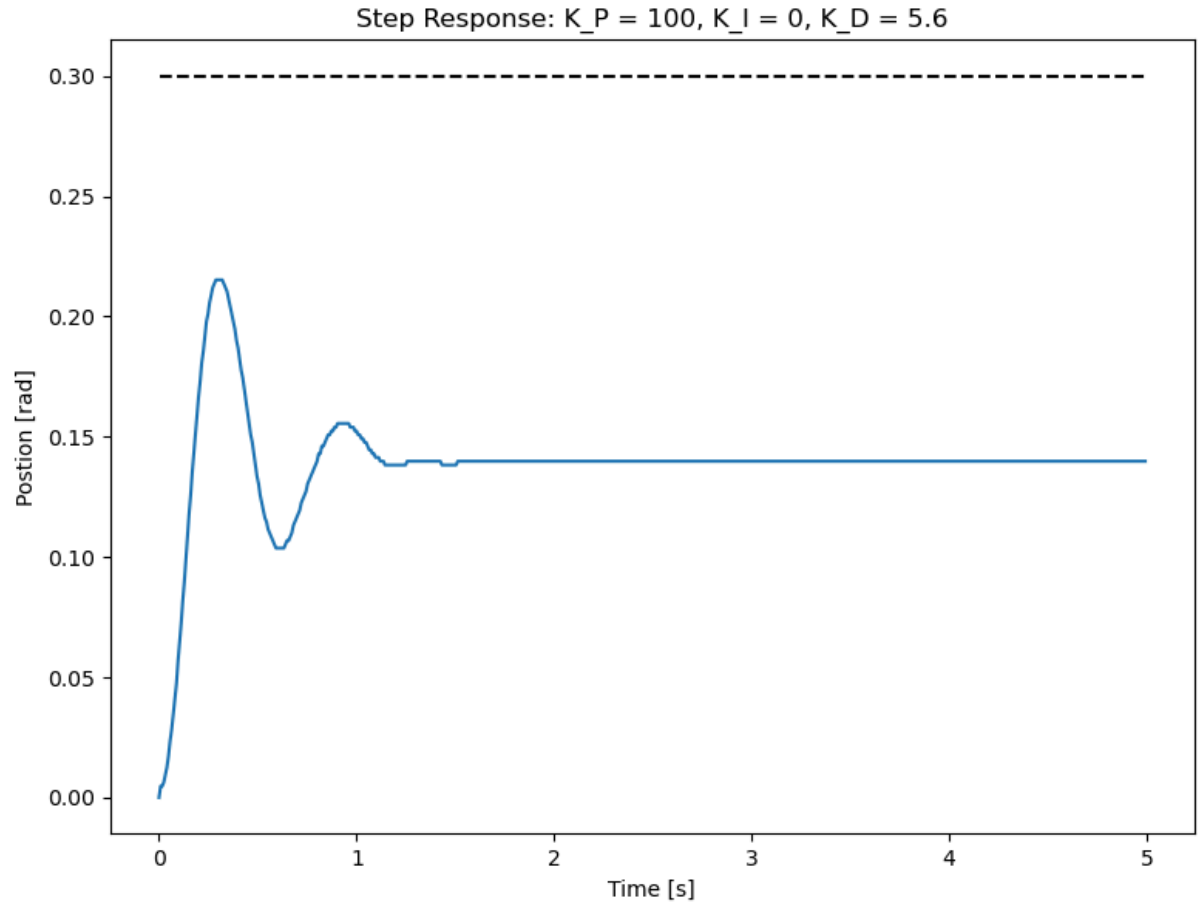

Controller Output
Voltage percent
100
75
50
25
0
−25
−50
−75
−100
P Part
I Part
D Part
Total
0
1
2
3
4
5
Time [s]

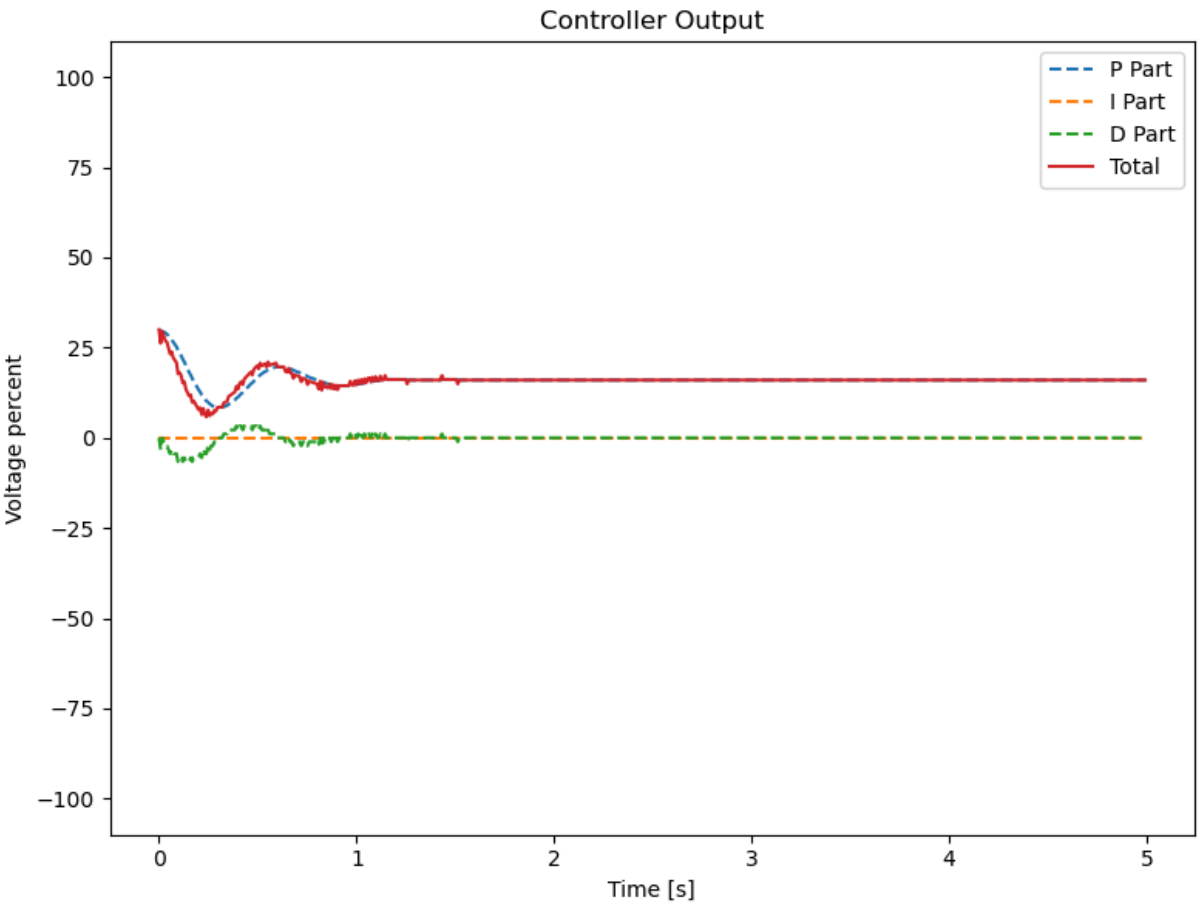
In [24]:
```
"run_step(100, 0, 5.6, 0.3, 5000, "data1.csv")"
```


Step Response: K_P = 100, K_I = 0, K_D = 5.6
Postion [rad]
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
1
2
3
4
5
Time [s]

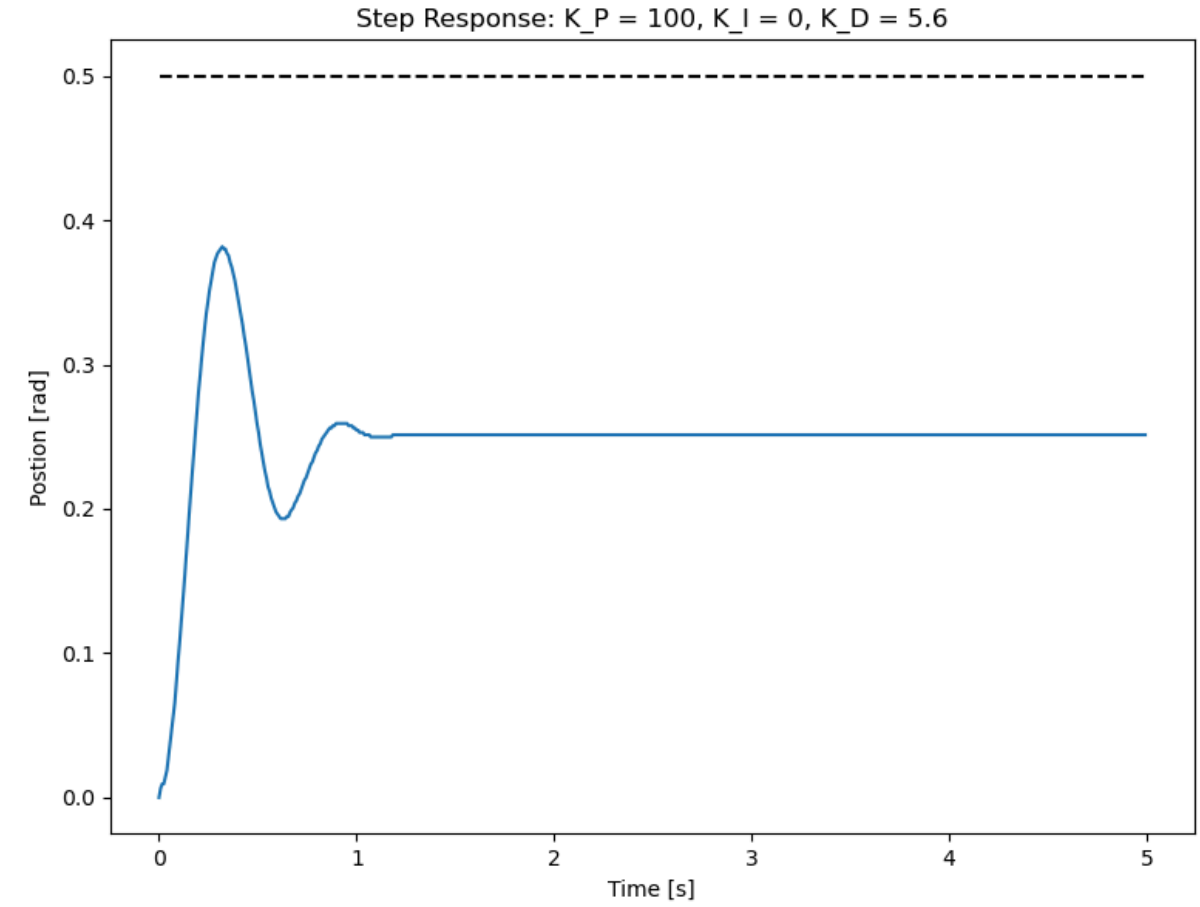

Controller Output
P Part
I Part
D Part
Total
Voltage percent
100
75
50
25
0
−25
−50
−75
−100
0
1
2
3
4
5
Time [s]

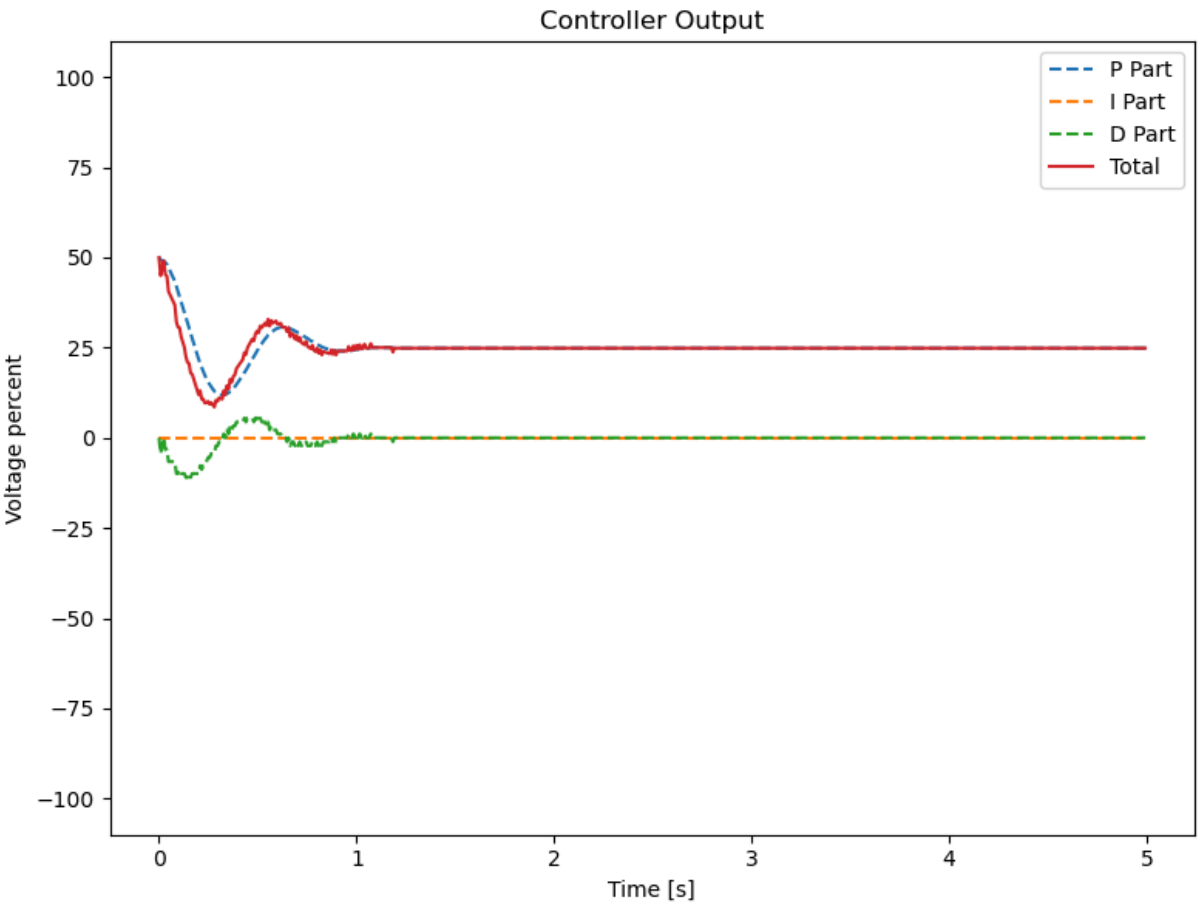
In [26]:
```
"run_step(100, 0, 5.6, 0.3, 5000, "data1.csv")"
```


Step Response: K_P = 100, K_I = 0, K_D = 5.6
Postion [rad]
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
1
2
3
4
5
Time [s]

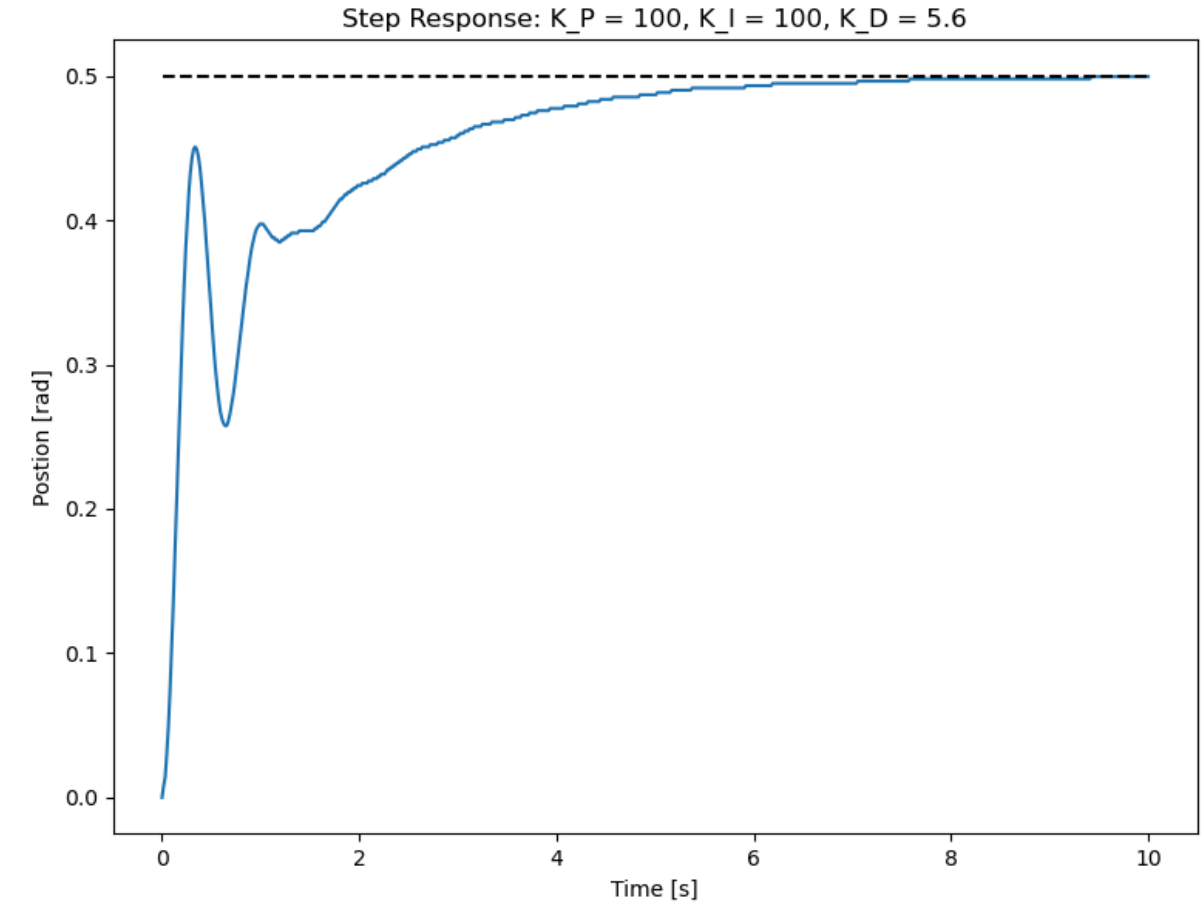

Controller Output
P Part
I Part
D Part
Total
Voltage percent
100
75
50
25
0
−25
−50
−75
−100
0
1
2
3
4
5
Time [s]

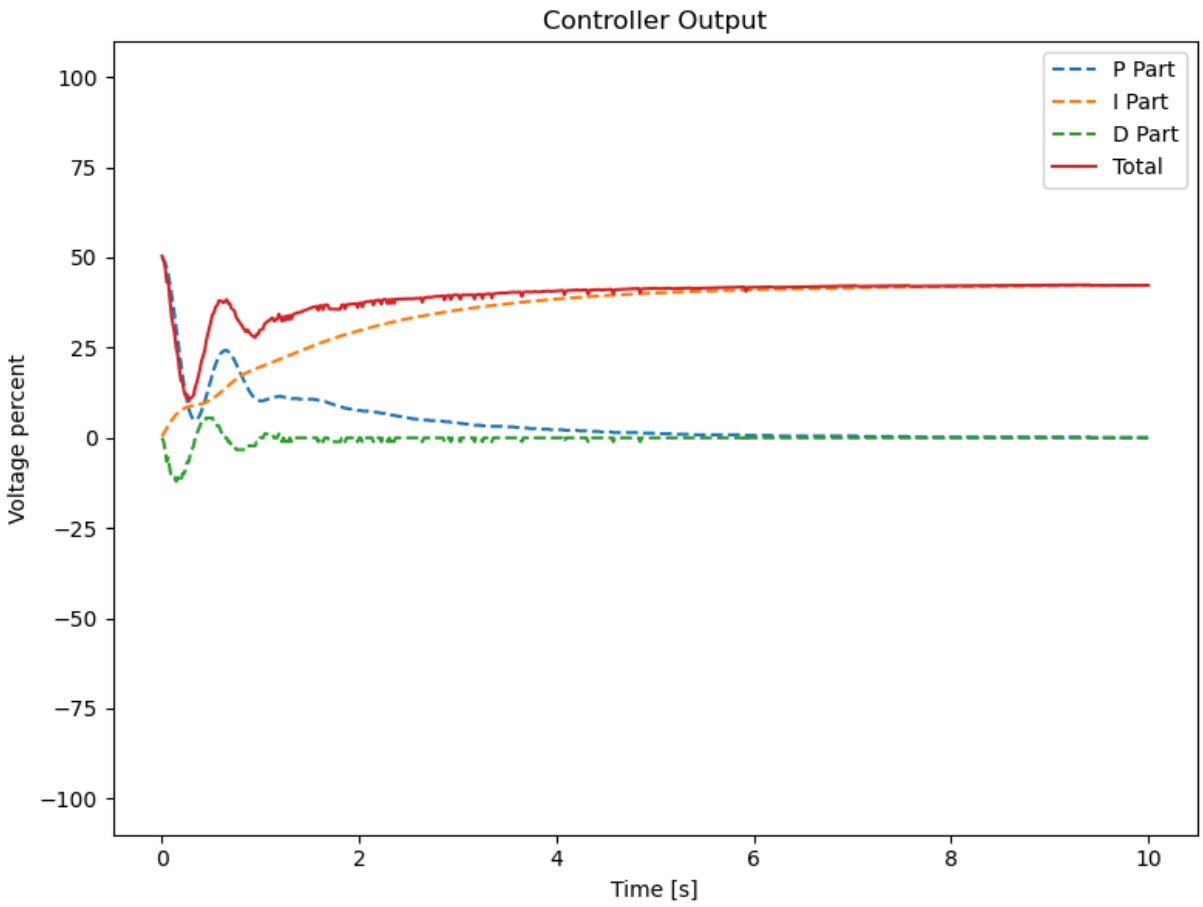
In [29]:
```
"run_step(100, 0, 5.6, 0.5, 5000, "data1.csv")"
```


Step Response: K_P = 100, K_I = 0, K_D = 5.6
Postion [rad]
Time [s]
0.0
0.1
0.2
0.3
0.4
0.5
0
1
2
3
4
5

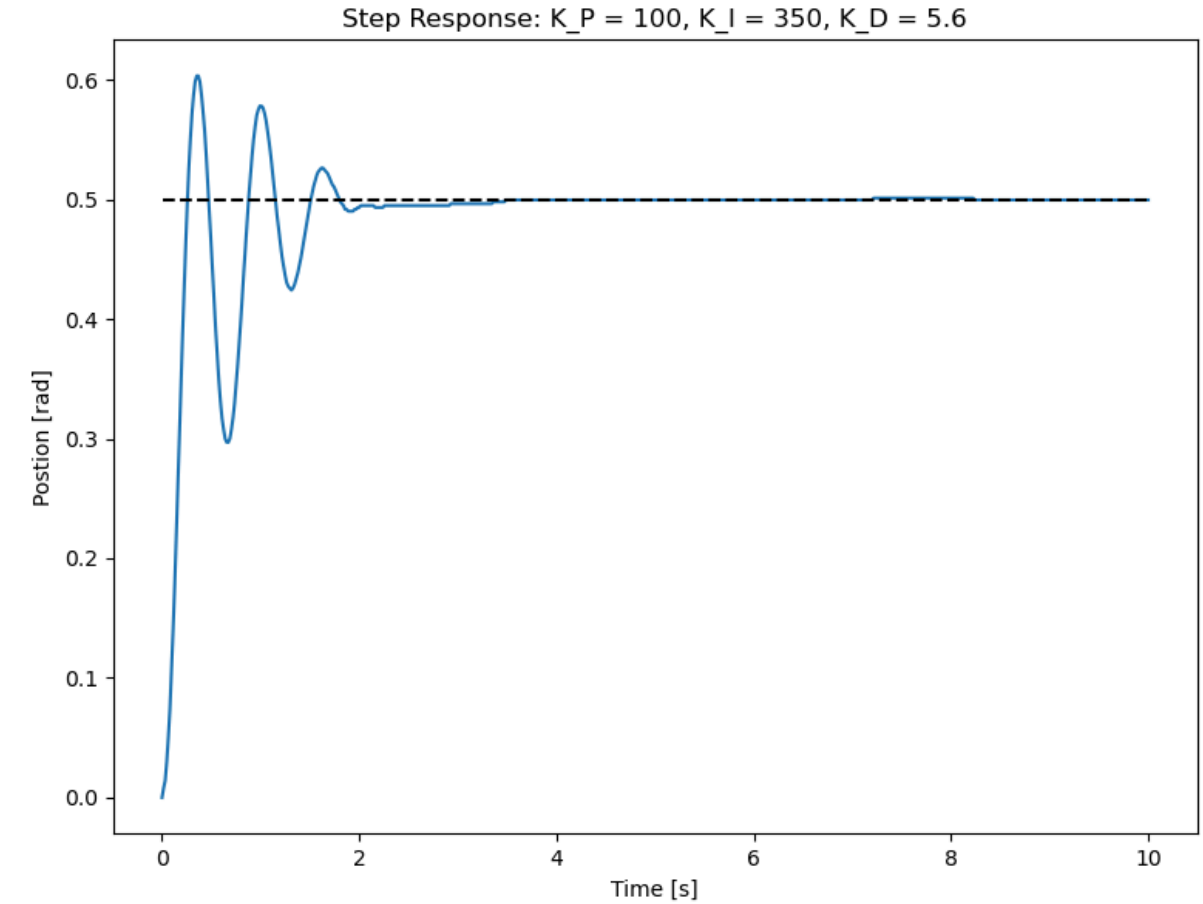

Controller Output
Voltage percent
Time [s]
P Part
I Part
D Part
Total
100
75
50
25
0
−25
−50
−75
−100
0
1
2
3
4
5

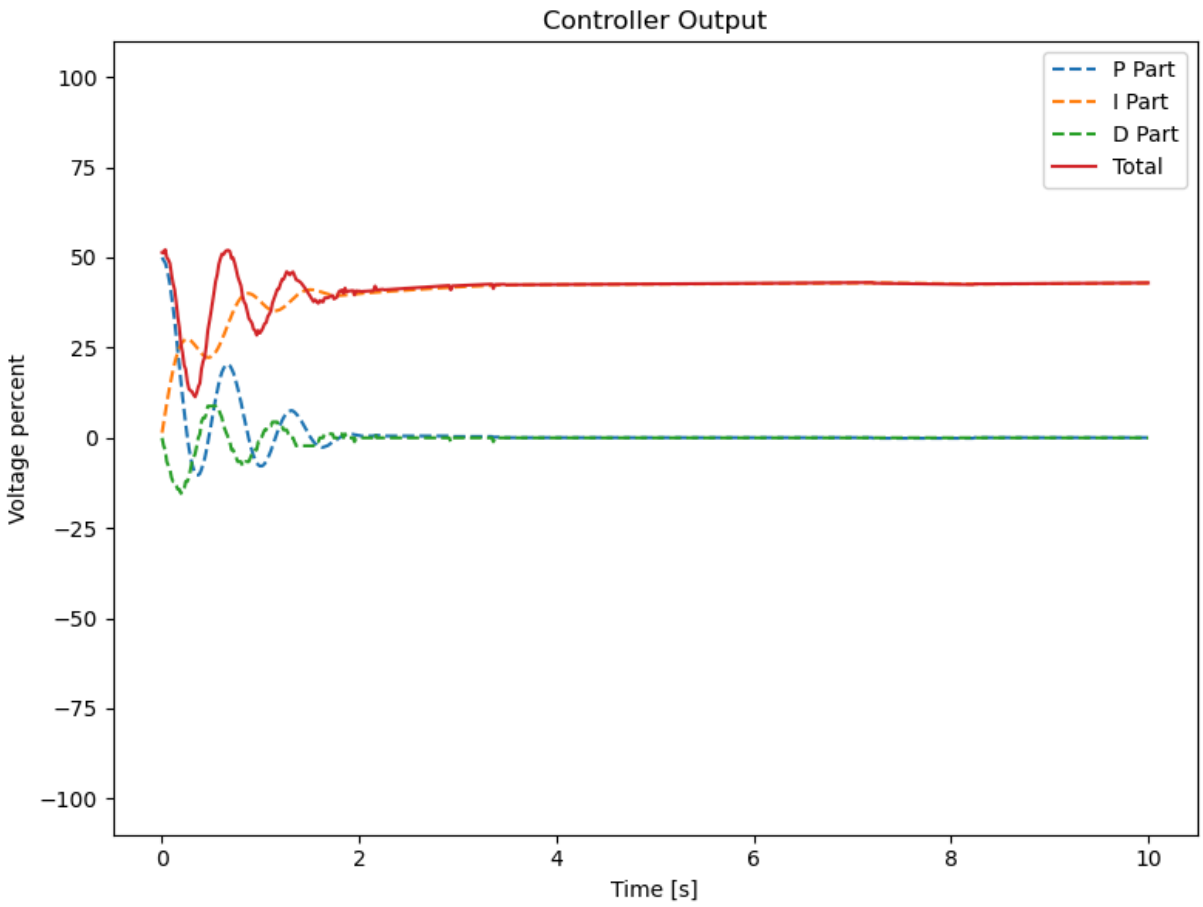
In [30]:
```
run_step(100, 100, 5.6, 0.5, 10000, "data1.csv")
```


Step Response: K_P = 100, K_I = 100, K_D = 5.6
Postion [rad]
Time [s]
0.0
0.1
0.2
0.3
0.4
0.5
0
2
4
6
8
10

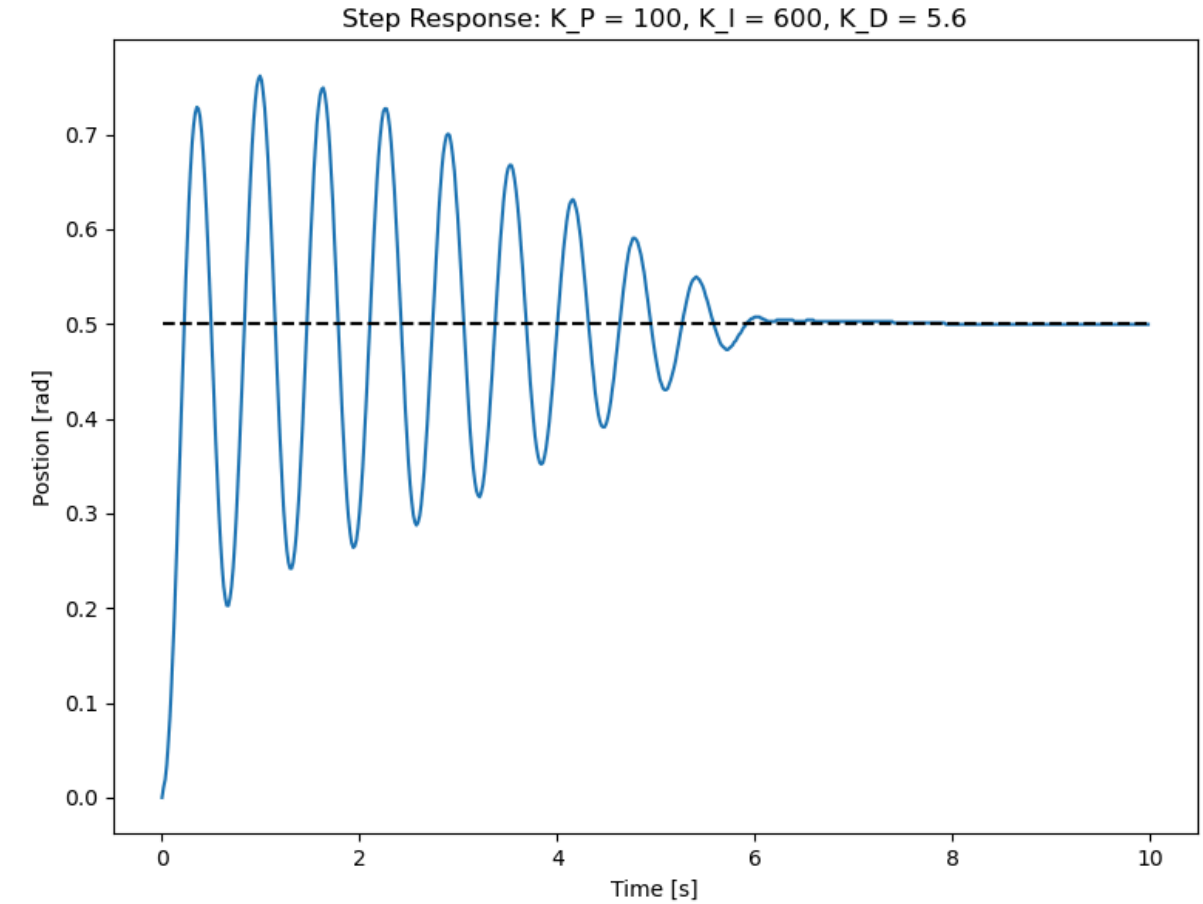

Controller Output
Voltage percent
Time [s]
P Part
I Part
D Part
Total
100
75
50
25
0
−25
−50
−75
−100
0
2
4
6
8
10

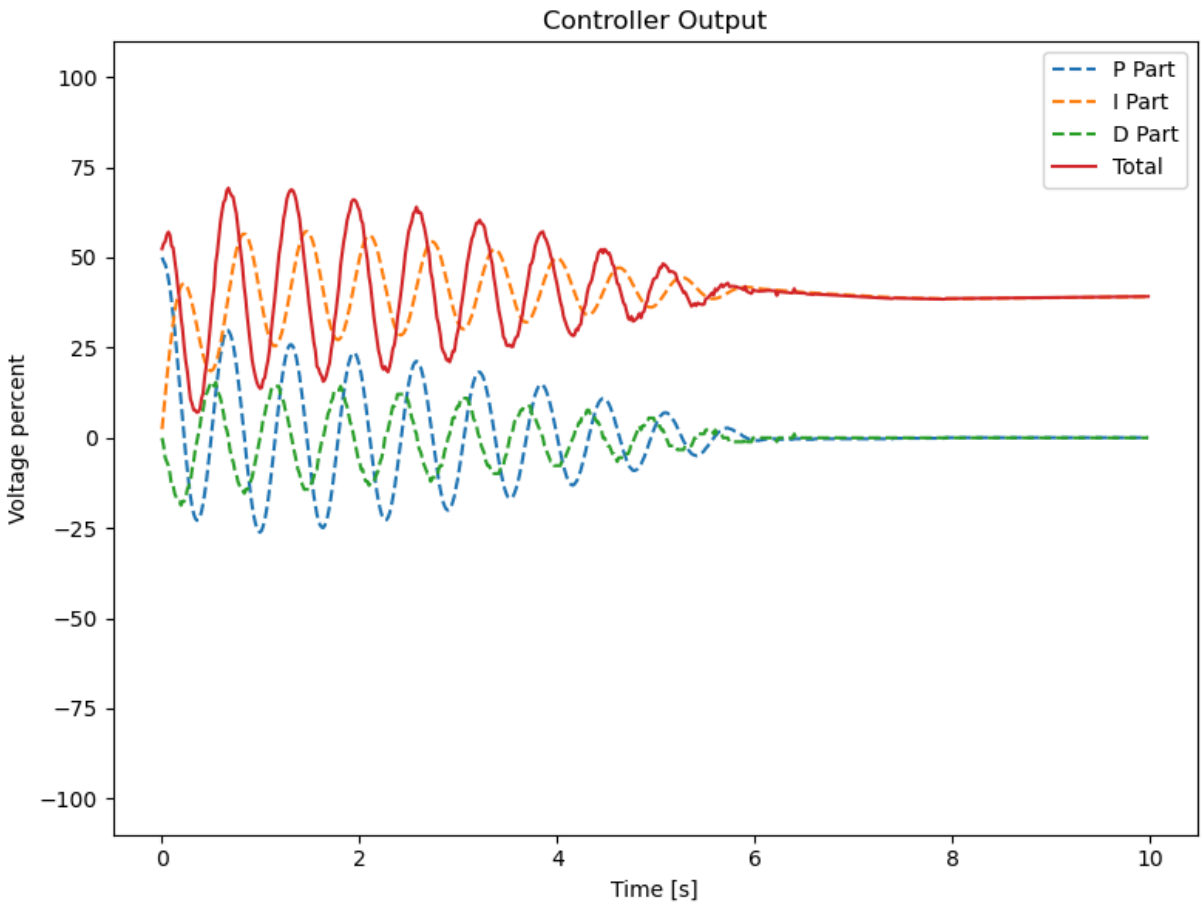
In [31]:
```
run_step(100, 350, 5.6, 0.5, 10000, "data1.csv")
```


Step Response: K_P = 100, K_I = 350, K_D = 5.6
Postion [rad]
Time [s]
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0
2
4
6
8
10


Controller Output
Voltage percent
Time [s]
P Part
I Part
D Part
Total
100
75
50
25
0
−25
−50
−75
−100
0
2
4
6
8
10

In [32]:
```
run_step(100, 600, 5.6, 0.5, 10000, "data1.csv")
```


Step Response: K_P = 100, K_I = 600, K_D = 5.6
Postion [rad]
Time [s]
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0
2
4
6
8
10


Controller Output
Voltage percent
Time [s]
P Part
I Part
D Part
Total
100
75
50
25
0
−25
−50
−75
−100
0
2
4
6
8
10

In [ ]: